

■ BEST Iași • Mentorat 2026

Training Web

De la zero la site live in 2 ore

HTML • CSS • JavaScript • Deploy • SEO • Lighthouse

Suport pentru participanti – 6 faze, fiecare cu aplicatie practica



Partea 0 | Inainte de training – pre-rechizite tehnice (5 min)



Faza 1 | **HTML** – structura, semantica, accesibilitate



Faza 2 | **CSS** – stilizare, layout, responsivitate



Faza 3 | **JavaScript** – interactivitate, manipulare DOM



Faza 4 | **Deploy** – versionare git, hosting GitHub Pages



Faza 5 | **SEO + Open Graph** – metadate pentru cautare si retele sociale



Faza 6 | **Lighthouse** – audit automatizat, Core Web Vitals



Resurse & urmatorii pasi – aplicatii interactive, tutoriale, comunitati

Pachetul servește atât ca suport în timpul training-ului, cât și ca referință de aprofundare ulterioară.

Structura manualului

Manualul prezent contine 7 capitole tematice (sase faze ale training-ului plus un capitol final cu resurse), structurate uniform pentru consultare facila. Fiecare capitol primeste o culoare distincta, vizibila in bara verticala stanga a blocurilor de cod si in numerotarea sectiunilor.

STRUCTURA UNUI CAPITOL

- **Teorie sintetica** – concepte, sintaxa, referinte la standarde.
- **Bloc de cod** – fragmente reutilizabile direct prin copiere.
-  **Aplicatie practica** – platforma online cu sarcina de exersare (5–10 minute).
-  **Resurse pentru aprofundare** – bibliografie cu linkuri.

CONVENTII VIZUALE

- Blocurile de cod sunt marcate printr-o bara verticala colorata in stanga, in culoarea capitolului.
- Toate URL-urile sunt clicabile in vizualizatorul PDF.
- Iconitele       identifica vizual fiecare capitol.

Proiectul fir roșu

Construim împreuna portofoliul Mariei Popescu.

Manualul nu prezinta concepte izolate. Pe parcursul celor 6 faze, fiecare strat se adauga la o pagina concreta – portofoliul Mariei Popescu, studenta CS in Iasi. La final, fiecare participant aplica acelasi pattern pentru pagina sa proprie.

Evoluția paginii pe parcursul training-ului:

Faza 1 – HTML. Text negru pe fundal alb, ca un document Word minimal. Structura semantica corecta in spate.

Pagina *exista* la nivel de informație, dar arata generic.

+ Faza 2 – CSS. Pagina capata identitate vizuala: paleta de culori, layout cu flexbox, header cu logo și navigare, hero cu CTA, dark mode.

Pagina *arata* ca un site profesional. Nu raspunde la interacțiuni.

+ Faza 3 – JavaScript. Hamburger menu funcțional pe mobil, smooth scroll la link-uri, formular care nu reincarca pagina.

Pagina *raspunde* la utilizator. Ramane offline, accesibila doar local.

+ Faza 4 – Deploy. Site-ul devine accesibil la maria.github.io/portfolio. URL real, share-uibil oriunde.

Pagina *exista in lume*. Dar oamenii inca nu o pot gasi.

+ Faza 5 – SEO. Cand cineva paste-uieste link-ul pe WhatsApp, apare card cu titlu, descriere si imagine. Google indexeaza pagina corect.

Pagina *e descoperibila*. Tehnic poate fi inca rafinata.

+ Faza 6 – Lighthouse. Audit pe 4 categorii. Scor 95+ pe Performance, Accessibility, Best Practices, SEO.

Pagina *e profesionala*. Standardul pe care firmele il cer la angajare.

✓ PE SCURT

Fiecare faza adauga o capacitate noua paginii. La final, pagina ta personala (cu numele tau in URL) trece toate verificarile pe care un dezvoltator profesional le-ar face.

Cuprins

Capitole, în ordinea recomandată.

Faza 1 – HTML	4
Faza 2 – CSS	13
Faza 3 – JavaScript	25
Faza 4 – Deploy	33
Faza 5 – SEO + Open Graph	39
Faza 6 – Lighthouse	45
Resurse & următorii pași	51
Anexe – Glosar, Checklist, Erori comune	55

Faza 1 – HTML

Structura documentului. Tag-uri semantice. Accesibilitate.

HTML (*HyperText Markup Language*) este limbajul de marcare folosit pentru descrierea **structurii** unui document web. Browserul citește documentul de sus în jos, construiește în memorie un arbore de noduri (DOM-ul), și afișează pagina pe ecran.

În arhitectura unei pagini web, responsabilitățile sunt împartite între trei tehnologii:

- **HTML** – ce conține pagina (titluri, paragrafe, imagini, formulare).
- **CSS** – cum arată pagina (culori, dimensiuni, spațiere, layout).
- **JavaScript** – cum se comportă pagina (răspuns la click, animații, validare).

Capitolul definește anatomia minimală, tag-urile semantice principale, atributele obligatorii pentru accesibilitate, și patternurile uzuale pentru linkuri și formulare.

🕒 CE ÎNVEȚI

După capitolul asta vei ști:

- Scrie boilerplate-ul HTML5 corect, cu cele 3 meta tags obligatorii.
- Distinge tag-urile semantice de `<div>` și alege tag-ul potrivit per zonă.
- Construiește formulare accesibile cu `<label for>` și `<input id>`.
- Aplica atributele `alt`, `lang`, `href` cu valori corecte conform WCAG.
- Recunoaște pattern-urile pentru linkuri externe sigure (`rel="noopener"`).

❓ DE CE

HTML este temelia oricărei pagini web. Browserele, cititoarele de ecran și motoarele de căutare interpretează pagina pornind de la structura HTML. O structură semantică greșită conduce la trei consecințe directe: SEO penalizat, persoanele cu dizabilități nu pot folosi pagina, codul devine ilizibil după câteva luni. Toate fazele următoare (CSS, JS, deploy, audit) presupun că structura HTML este corectă.

1. Anatomia documentului

Un document HTML5 conține cinci elemente structurale obligatorii. Pe linia 1, declarația `<!DOCTYPE html>` indică browserului că documentul respectă specificația HTML5; în absența sa, browserul intră în modul „quirks” care reproduce comportamentul browserelor din anii 2000.

Listing 1: Boilerplate HTML5 conform specificației W3C

```
1 <!DOCTYPE html>
2 <html lang="ro">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
      initial-scale=1.0">
```

```
6 <title>Numele Prenumele -- Student CS, Iasi</title>
7 </head>
8 <body>
9   <!-- continut vizibil -->
10 </body>
11 </html>
```

- `<html lang="ro">` – elementul radacina. Atributul `lang` influenteaza pronuntia in cititoare de ecran, ranking-ul in cautari localizate, si detectia limbii sursa pentru traducere automata.
- `<head>` – contine metadatele paginii. Continutul nu apare pe ecran, dar este consumat de browser, motoare de cautare si platforme sociale.
- `<body>` – contine elementele vizibile pe ecran. Tot ce vede utilizatorul se afla in interiorul acestui tag.

2. Sintaxa tag-urilor

Un element HTML are una sau mai multe parti: tag de deschidere, continut, tag de inchidere. Atributele se declara in tag-ul de deschidere, in formatul `nume="valoare"`.

Listing 2: Tipuri de tag-uri

```
1 <!-- Tag container: are continut intre deschidere si inchidere -->
2 <p>Acesta este un paragraf.</p>
3
4 <!-- Tag void: nu are continut, nu se inchide -->
5 
6
7 <!-- Tag cu attribute multiple -->
8 <a href="https://best-is.ro" target="_blank" rel="noopener">BEST</a>
```

Tag-urile void cuprind: `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`.

3. Metadate obligatorii in `<head>`

Trei elemente sunt considerate obligatorii in orice pagina moderna:

1. `<meta charset="UTF-8">` – setul de caractere. UTF-8 acopera toate alfabetele moderne, inclusiv diacriticele romanesti.
2. `<meta name="viewport"...>` – controleaza redarea pe dispozitive mobile. Valoarea `width=device-width, initial-scale=1.0` instruceaza browserul sa pastreze latimea paginii egala cu cea a ecranului si sa nu zoom-eze initial.
3. `<title>` – apare in tab-ul browserului, in bookmark-uri, si ca titlu in rezultatele cautarii.

⚠ ATENȚIE

Conform datelor StatCounter (2024), peste 60% din traficul web global provine de pe dispozitive mobile. Lipsa meta tag-ului `viewport` afecteaza direct experienta majoritatii utilizatorilor, generand o pagina nescalata, cu zoom necesar pentru lectura.

4. Tag-uri semantice vs <div> soup

Tag-urile semantice descriu *rolul* conținutului, nu doar aspectul vizual. Browserul, motoarele de căutare și tehnologiile asistive (cititoare de ecran, navigatoare prin tastatură) pot interpreta automat structura paginii când tag-urile semantice sunt folosite.

✘ ASA NU – „DIV SOUP”

```
<div class="header">
  <div class="logo">Site</div>
  <div class="menu">
    <div class="link"><a href="/">Acasa</a></div>
  </div>
</div>
<div class="content">
  <div class="hero">
    <div class="title">Salut</div>
  </div>
</div>
<div class="footer">2026</div>
```

Probleme. Browserul și cititoarele de ecran nu știu ce reprezintă fiecare <div>. Navigația rapidă prin landmarks este imposibilă. SEO penalizat pentru lipsa de structură.

✔ ASA DA – SEMANTIC

```
<header>
  <a class="logo" href="/">Site</a>
  <nav>
    <a href="/">Acasa</a>
  </nav>
</header>
<main>
  <section class="hero">
    <h1>Salut</h1>
  </section>
</main>
<footer>2026</footer>
```

Beneficii. Cititoarele de ecran oferă comenzi rapide pentru sărit la <header>, <nav>, <main>, <footer>. Google identifică zonele principale fără analiză euristică. Codul este mai ușor de citit după 6 luni.

4.1 Repertoriul tag-urilor semantice

- <header> – antetul unei pagini sau al unei secțiuni.
- <nav> – bara de navigație principală.
- <main> – conținutul unic al paginii (un singur <main> per pagină).
- <section> – grup tematic de conținut, de obicei cu un <h2>.

- `<article>` – unitate de conținut auto-suficientă (post de blog, comentariu, card produs).
- `<aside>` – conținut tangential (sidebar, note de subsol vizuale).
- `<footer>` – subsolul unei pagini sau al unei secțiuni.
- `<figure>` + `<figcaption>` – imagine cu legenda.

5. Ierarhia titlurilor

Specificația HTML5 definește șase niveluri de titluri, de la `<h1>` la `<h6>`. Cititoarele de ecran generează un *outline* al paginii pe baza acestei ierarhii; un utilizator nevăzător parcurge documentul saltând între titluri exact ca un cititor obișnuit când parcurge un cuprins.

- Un singur `<h1>` per pagină, descriind tema centrală.
- Subsecțiunile principale folosesc `<h2>`; sub-subsecțiunile, `<h3>`.
- Nivelurile nu se sar (`<h1>` urmat direct de `<h3>` este o eroare structurală).
- Marimea vizuală se controlează prin CSS, nu prin alegerea unui nivel diferit.

✗ ASA NU – STILUL ALES PRIN NIVEL

```
<h1>Titlul mare</h1>
<h4>Subtitlu, am ales h4 ca arata mai mic</h4>
<h2>Secțiune</h2>
```

✓ ASA DA – STILUL ALES PRIN CSS

```
<h1>Titlul mare</h1>
<h2 class="subtitle">Subtitlu</h2>
<h2>Secțiune</h2>
```

```
.subtitle { font-size: 1rem; font-weight: normal; }
```

6. Linkuri și navigare

Elementul `<a>` (anchor) creează un hyperlink. Atributul `href` accepta mai multe forme:

Listing 3: Tipuri de URL-uri

```
1 <!-- URL absolut -->
2 <a href="https://best-is.ro">BEST Iasi</a>
3
4 <!-- URL relativ -->
5 <a href="contact.html">Contact</a>
6 <a href="../images/logo.png">Logo</a>
7
8 <!-- Ancora interna -->
9 <a href="#contact">Sari la contact</a>
10
```



```

11 <!-- Adresa email / numar de telefon -->
12 <a href="mailto:hello@best-is.ro">Email</a>
13 <a href="tel:+40123456789">Telefon</a>

```

6.1 Linkuri externe – target si rel

Deschiderea unui link extern intr-un tab nou se face cu `target="_blank"`. Atributul aduce un risc de securitate: pagina destinatie poate manipula tab-ul sursa prin `window.opener`. Combinatia `rel="noopener noreferrer"` elimina riscul.

✖ ASA NU

```
<a href="https://extern.ro" target="_blank">Extern</a>
```

Problema: pagina externa poate redirectiona tab-ul tau initial catre o pagina de phishing.

✔ ASA DA

```
<a href="https://extern.ro" target="_blank" rel="noopener noreferrer">Extern</a>
```

Beneficiu: `noopener` blocheaza accesul la `window.opener`; `noreferrer` ascunde URL-ul sursa.

7. Formulare accesibile

Fiecare element `<input>` trebuie asociat cu un `<label>`. Asocierea se realizeaza prin perechea atribut-valoare `for/id`: valoarea atributului `for` de pe `<label>` este identica cu valoarea atributului `id` de pe `<input>`.

✖ ASA NU – PLACEHOLDER CA SUBSTITUT PENTRU LABEL

```
<input type="email" placeholder="Email">
```

Probleme. (1) Cititoarele de ecran nu anunta placeholder-ul ca eticheta. (2) Placeholder-ul dispare la tastare, lasand utilizatorul fara context. (3) Contrast scazut pe textul de placeholder.

✔ ASA DA

```

<label for="email">Email</label>
<input type="email" id="email" name="email" required>

```

Beneficii. Cititoarele anunta corect; clic-ul pe text focuseaza inputul; eticheta ramane vizibila.

7.1 Tipuri de input frecvente

Atributul `type` controleaza interfata oferita de browser si validarea. Pe mobil, tipul determina tastatura afisata.

- `type="text"` – text generic.
- `type="email"` – validare format email; tastatura cu @ pe mobil.
- `type="tel"` – tastatura numerica pe mobil; nu valideaza formatul (variaza pe tari).
- `type="number"` – accepta doar cifre; arrows pe desktop.

- `type="date"` – date picker nativ.
- `type="password"` – ascunde caracterele.
- `type="url"` – validare format URL.
- `type="search"` – buton clear in unele browsere.

💡 PONT

Browserul valideaza nativ inputurile: campurile cu atributul `required` blocheaza submit-ul daca sunt goale; `type="email"` respinge valori care nu contin @. Aceasta validare **nu inlocuieste** validarea pe server, dar imbunatateste experienta utilizatorului inainte de a trimite cererea.

8. Atribute esentiale

- `href` pe `<a>` – URL-ul tinta.
- `src` pe `<img>` – calea catre imagine.
- `alt` pe `<img>` – text descriptiv. Obligativ conform WCAG 2.1 (criteriul 1.1.1 *Non-text Content*).
- `type` pe `<input>` – determina validarea si interfata.
- `required` pe `<input>` – activeaza validarea nativa.
- `lang` pe `<html>` – limba documentului.
- `title` pe orice element – tooltip la hover (uz limitat, nu inlocuieste `aria-label`).

8.1 Calitatea textului alt

Textul `alt` descrie ce vede imaginea, in contextul paginii. Scopul: utilizatorul fara acces vizual primeste informatia echivalenta.

✘ ASA NU – ALT NESEMNIFICATIV SAU SPAM SEO

```



```

✔ ASA DA – ALT DESCRIPTIV SI SPECIFIC

```

```

Exceptie: pentru imagini pur decorative (fundal, separator, ornament), `alt=""` este corect – cititorul de ecran le ignora.

9. Tag-uri suplimentare – mențiuni rapide

Pentru proiecte mai complexe, HTML5 ofera elemente specializate care nu intra in scopul fazei curente, dar merita cunoscute la nivel de existență:

- `<table>` cu `<thead>`, `<tbody>`, `<tfoot>`, `<caption>` – pentru date tabulare reale (nu pentru layout). Atributele `scope`, `headers` asigura accesibilitatea.
- `<video>` si `<audio>` – mediafile native, fara nevoie de plugin-uri (Flash, Silverlight au murit). Atribute: `controls`, `autoplay`, `muted`, `loop`, `poster` (pentru video).
- `<canvas>` – suprafața 2D pentru desenare programatica cu JavaScript. Folosit pentru jocuri, vizualizari, animații complexe.
- `<details>` si `<summary>` – accordeon nativ, fara JavaScript.
- `<dialog>` – modal nativ, suportat in toate browserele moderne.
- `<picture>` – container pentru `<img>` cu surse alternative pe formate sau dimensiuni.

Documentația exhaustiva: MDN – HTML elements reference (link in resursele de la finalul capitoului).

10. Comentarii si caractere speciale

Comentariile HTML nu apar in pagina dar raman vizibile in *View Source*. Sintaxa: `<!-- text -->`.

Caracterele rezervate (`<`, `>`, `&`) trebuie scrise prin entitati pentru a nu fi interpretate ca markup:

- `<`; – `<`
- `>`; – `>`
- `&`; – `&`
- `©`; – (c)
- ` `; – spatiu nedespartibil

Verificare cunoștințe

☰ PROVOCARE

1. Care este diferența funcțională între `<div class="header">` și `<header>`?
2. De ce este nevoie de `<meta name="viewport">` pentru afișarea pe mobil?
3. *Spot the issue:* `<input type="email" placeholder="Email">`.
4. Cate elemente `<h1>` pot exista intr-o pagina conform specificației HTML5?
5. Care sunt cele doua atribute necesare cand un link extern se deschide intr-un tab nou?

Recapitulare

✓ FAZA 1 – HTML

- Anatomia: `<!DOCTYPE html>` → `<html lang>` → `<head>` (metadate) → `<body>` (continut).
- Trei meta tags obligatorii: `charset`, `viewport`, `<title>`.
- Tag-uri semantice in locul `<div>`: `<header>`, `<main>`, `<section>`, `<footer>`, `<nav>`, `<article>`, `<aside>`.
- Un singur `<h1>` per pagina; ierarhia titlurilor nu admite salturi de nivel.
- Forms accesibile: `<label for="X">` corespunde cu `<input id="X">`.
- Attribute obligatorii pentru accesibilitate: `alt` pe `<img>`, `lang` pe `<html>`.
- Linkuri externe in tab nou: combinatia `target="_blank" + rel="noopener noreferrer"`.



🛠️ APLICAȚIE PRACTICĂ

Platforma: CodePen (codepen.io). URL-ul exact este distribuit de trainer in chat.

Activitatea. Documentul de pornire contine o structura non-semantica (`<div>` folosit excesiv), atribute `alt` lipsa pe imagini si elemente `<input>` fara `<label>` asociat. Sarcina este de a transforma documentul intr-o varianta semantica si conformanta cu standardele de accesibilitate.

Criterii de validare:

- Folosirea tag-urilor `<header>`, `<main>`, `<section>`, `<footer>` in locul `<div>`.
- Prezenta atributului `alt` pe toate elementele `<img>`.
- Asocierea `<label for>` – `<input id>` pentru fiecare camp de formular.
- Existenta unui singur element `<h1>` per pagina.
- Linkuri externe au `target="_blank" + rel="noopener noreferrer"`.

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Mozilla Developer Network. *HTML elements reference*.
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element>
- The Odin Project. *HTML Foundations*.
<https://www.theodinproject.com/paths/foundations/courses/foundations#html-foundations>
- Google. *Learn HTML*. <https://web.dev/learn/html>
- HTML Reference. *Vizual cheatsheet de tag-uri*. <https://htmlreference.io>
- Google. *Learn Forms*. <https://web.dev/learn/forms>
- W3C. *Web Accessibility Initiative – WCAG*. <https://www.w3.org/WAI/standards-guidelines/wcag/>

→ **Urmeaza:** Capitolul 2 (CSS) imbraca structura HTML cu stil vizual. Aspectul, layout-ul și

responsivitatea se construiesc plecand de la pagina semantica generata aici.

Faza 2 – CSS

Stilizare. Layout. Responsivitate.

CSS (*Cascading Style Sheets*) controlează aspectul vizual al documentelor HTML. Numele provine de la *cascada*: când mai multe reguli vizează același element, browserul aplică un algoritm de prioritate (cascada) pentru a alege valoarea finală.

Capitolul prezintă selectorii, modelul de cutie, unitățile de măsură, variabilele CSS, sistemele de layout flexbox și grid, abordarea *mobile-first*, și suportul nativ pentru tema dark/light.

🎯 CE ÎNVEȚI

După capitolul asta vei ști:

- Folosește selectorii CSS și înțelege specificitatea ca triplet (*id*, *clasa*, *tag*).
- Aplică modelul de cutie cu `box-sizing: border-box` pentru calcule predictibile.
- Definește variabile CSS în `:root` și le refolosește prin `var()`.
- Construiește layout-uri cu flexbox (6 proprietăți cheie).
- Scrie reguli mobile-first cu `@media (min-width: ...)`.
- Poziționează elemente cu `position` și creează header sticky.
- Aplică animații cu `transition` și `@keyframes`, respectând `prefers-reduced-motion`.

❓ DE CE

CSS face diferența între pagina-document și pagina-produs. Aceeași structură HTML poate părea neprofesională sau profesională, în funcție de stilizare. Mai mult: CSS-ul prost arhitecturat (cu *!important*, specificitate ridicată, valori absolute) generează ore de depanare. Convențiile prezentate aici (variabile, mobile-first, flexbox) sunt cele aplicate în practica industrială.

ℹ️ PRE-RECHIZITE

Capitolul presupune înțelegerea tag-urilor HTML semantice și a structurii `<head>/<body>` (Faza 1).

1. Conectarea fișierului CSS la HTML

CSS poate fi inclus în trei moduri. Convenția recomandată este externalizarea într-un fișier separat:

Listing 4: Stilurile externe (recomandat)

```
1 <head>
2   <link rel="stylesheet" href="style.css">
3 </head>
```

⊗ ASA NU – STILURI INLINE IMPRASTIATE

```
<button style="background: blue; color: white; padding:
```

```
10px;">Click</button>
```

Probleme. Imposibil de reutilizat; stilul nu poate fi schimbat global; specificitatea ridicata creeaza conflicte; pagina HTML devine ilizibila.

✓ ASA DA – CLASA CU STIL EXTERN

```
<button class="cta">Click</button>
```

In `style.css`: `.cta { background: blue; color: white; padding: 10px; }`

2. Selectorii

Selectorii determina elementele asupra carora se aplica un set de reguli. Categoriile principale:

Listing 5: Selectorii uzuali

```
1  /* Selector de tip: vizeaza toate elementele <h1> */
2  h1 { color: navy; }
3
4  /* Selector de clasa: reutilizabil, prefixat cu . */
5  .cta { background: orange; }
6
7  /* Selector de id: unic pe pagina, prefixat cu # */
8  #contact { padding: 2rem; }
9
10 /* Selector de atribut */
11 input[type="email"] { border-color: blue; }
12
13 /* Combinator descendent: <a> oriunde sub <nav> */
14 nav a { text-decoration: none; }
15
16 /* Combinator copil direct: <li> direct sub <ul> */
17 ul > li { margin-bottom: 0.5rem; }
18
19 /* Combinator frate adiacent: <p> imediat dupa <h2> */
20 h2 + p { margin-top: 0; }
21
22 /* Pseudo-clase: stari ale elementului */
23 .cta:hover { background: red; }
24 .cta:focus-visible { outline: 2px solid blue; }
25 .cta:disabled { opacity: 0.5; }
26
27 /* Pseudo-elemente: parti generate */
28 p::first-letter { font-size: 2em; }
```

2.1 Specificitate – cum se rezolva conflictele

Cand mai multe reguli vizeaza acelasi element, browserul calculeaza o specificitate pentru fiecare si o aplica pe cea cu valoare maxima. Calculul foloseste un triplet (a, b, c):

- **a** – numărul de id-uri.
- **b** – numărul de clase, attribute, pseudo-clase.
- **c** – numărul de tag-uri și pseudo-elemente.

Comparatia se face lexicografic: $(0,1,0)$ pierde fata de $(1,0,0)$ indiferent de **b** și **c**. Stilurile inline au specificitate $(1,0,0,0)$, peste orice selector. Cuvantul cheie `!important` suprascrie tot, dar este considerat un *antipattern* – indica faptul ca arhitectura CSS este compromisa.

✗ ASA NU – FOLOSIREA `!important` CA SOLUTIE

```
.btn { background: red !important; }
```

Problema. Cand alta regula are nevoie sa schimbe culoarea, va trebui si ea sa foloseasca `!important`, escaladand conflictul.

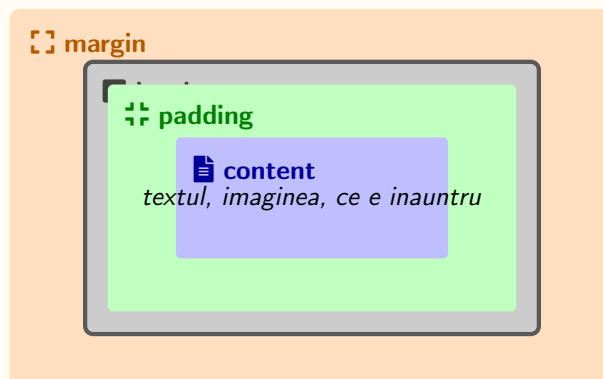
✓ ASA DA – SPECIFICITATE PROPORTIONALA

```
.btn { background: blue; } .btn.btn-danger { background: red; }
```

Solutie: clase modificador (`.btn-danger`) ridica specificitatea natural cu $(0,2,0)$ fata de $(0,1,0)$.

3. Modelul de cutie

Fiecare element redat de browser este o cutie compusa din patru straturi concentrice: *content*, *padding*, *border*, *margin*.



- **content** – zona ocupata de continutul efectiv (text, imagine).
- **padding** – spatiul dintre continut și chenar.
- **border** – chenarul.
- **margin** – spatiul exterior, între cutia curenta și elementele adiacente.

3.1 Implicit vs border-box

Comportamentul implicit (`box-sizing: content-box`) include în `width` doar continutul. Adaugarea de `padding` sau `border` maresta latimea totala. Acest comportament face calculele predispuse la erori.

✗ ASA NU – CONTENT-BOX, LATIMI IMPREVIZIBILE

```
.card {
```



```
width: 300px;
padding: 20px;
border: 1px solid #ccc;
/* Latime totala: 300 + 20*2 + 1*2 = 342px (oversize) */
}
```

✓ ASA DA – BORDER-BOX GLOBAL, LATIMI PREDICTIBILE

```
*, *::before, *::after { box-sizing: border-box; }

.card {
width: 300px;
padding: 20px;
border: 1px solid #ccc;
/* Latime totala: 300px (padding si border absorbite in width)
*/
}
```

4. Unitati de masura

CSS suporta zeci de unitati. In practica, doar 5-6 acopera majoritatea cazurilor:

- **px** – pixel absolut. Pentru border-uri si dimensiuni de detaliu.
- **rem** – relativ la font-size-ul `<html>` (implicit 16px). Pentru font-size si spatiere uniforma.
- **em** – relativ la font-size-ul elementului parinte. Util pentru padding-uri scaland cu textul.
- **%** – relativ la dimensiunea parintelui pe acea axa.
- **vw / vh** – 1% din latimea / inaltimea viewport-ului.
- **ch** – latimea unui caracter „0”. Util pentru `max-width` pe paragrafe.

💡 PONT

Pentru font-size si spatiere generala, **rem** este alegerea recomandata. Cand utilizatorul mareste font-size-ul implicit din browser (Settings → Appearance), tot site-ul scaleaza proportional. **px** pe font-size ignora aceasta preferinta.

Listing 6: Componere uzuala

```
1 :root {
2   font-size: 100%;           /* echivalent 16px in browserele
   standard */
3 }
4
5 body { font-size: 1rem; }    /* 16px */
6 h1   { font-size: 2.5rem; } /* 40px */
7 .container { max-width: 64rem; } /* 1024px */
8
```

```

9  p { max-width: 65ch; }           /* lungime optima de citire */
10
11  .hero {
12    min-height: 80vh;             /* 80% din inaltimea ferestrei */
13    padding: 2rem;
14  }

```

5. Variabile CSS (custom properties)

Variabilele CSS permit definirea valorilor o singura data si reutilizarea lor pe parcursul intregului document. Modificarea unei singure declaratii in `:root` se propaga automat in toate locurile in care variabila este folosita.

```

1  :root {
2    /* Paleta */
3    --primary: #0673BA;
4    --accent:  #FAA51E;
5    --bg:      #FFFCF6;
6    --text:    #1A1A1A;
7
8    /* Spatiere consistenta */
9    --space-1: 0.5rem;
10   --space-2: 1rem;
11   --space-3: 2rem;
12
13   /* Layout */
14   --max-width: 64rem;
15   --radius: 0.5rem;
16 }
17
18 body {
19   background: var(--bg);
20   color: var(--text);
21 }
22
23 h1 { color: var(--primary); }
24 .cta { background: var(--accent); }

```

Variabilele CSS sunt *scoped*: redefinirea intr-un selector descendent suprascrie valoarea pentru subarboare. Acest comportament permite teme contextuale fara duplicarea regulilor:

Listing 7: Tema schimbata local

```

1  .card-dark {
2    --bg:    #1F2937;
3    --text:  #F1F5F9;
4    /* toate copiii care folosesc var(--bg) si var(--text) preiau
5       noile valori */
6  }

```

6. Flexbox

Flexbox (*Flexible Box Layout*) este un model de layout uni-dimensional, util pentru distribuirea spatiului si alinierea continutului pe o axa (orizontala sau verticala).

Listing 8: Header cu logo si navigare aliniate

```
1 header {
2   display: flex;                /* activeaza flexbox */
3   flex-direction: row;          /* axa principala = orizontala
   (default) */
4   justify-content: space-between; /* distribuire pe axa principala
   */
5   align-items: center;          /* aliniere pe axa secundara */
6   flex-wrap: wrap;              /* trecere pe randul urmator daca
   nu incapa */
7   gap: var(--space-2);         /* spatiu intre copii */
8 }
```

6.1 Cele 6 proprietati pe care le folosesti

- `display: flex` – activeaza modelul.
- `flex-direction: row | column` – axa principala.
- `justify-content` – distributie pe axa principala. Valori uzuale: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`.
- `align-items` – aliniere pe axa secundara. Valori uzuale: `stretch` (default), `center`, `flex-start`, `flex-end`.
- `gap` – spatiu uniform intre copii. Inlocuieste `margin` pe copii.
- `flex: 1` (pe copii) – permite copilului sa creasca pentru a umple spatiul disponibil.

💡 PONT

Trick uzual: `display: flex; justify-content: center; align-items: center;` centreaza orice element in containerul sau, atat orizontal cat si vertical, in 3 linii.

7. CSS Grid (introducere scurta)

Grid este sistemul de layout bidimensional al CSS-ului. Spre deosebire de flexbox (uni-dimensional), grid permite controlul simultan al randurilor si coloanelor.

Listing 9: Galerie cu coloane responsive automat

```
1 .gallery {
2   display: grid;
3   grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
4   gap: var(--space-2);
5 }
```

Mecanism: `auto-fit` creeaza cate coloane incap; `minmax(240px, 1fr)` cere o latime minima de

240px si o expansiune egala (1fr) cand ramane spatiu. Rezultatul: galerie responsive fara media queries.

8. Mobile-first

Strategia *mobile-first* consta in scrierea regulilor pentru ecrane mici ca valori implicite, urmata de extinderea acestora pentru ecrane mai mari prin *media queries*:

```
1 /* Stiluri implicite, pentru mobil */
2 .nav { display: none; }
3 .nav-toggle { display: flex; }
4
5 /* Adaugiri pentru ecrane >= 768px */
6 @media (min-width: 768px) {
7   .nav { display: flex; }
8   .nav-toggle { display: none; }
9 }
```

✗ ASA NU – DESKTOP-FIRST CU MAX-WIDTH

```
.nav { display: flex; }
@media (max-width: 767px) {
  .nav { display: none; }
  .nav-toggle { display: flex; }
}
```

Problema. Mobilul descarca si proceseaza CSS conceput pentru desktop, apoi il anuleaza. Pierdere de performanta pe contextul cu cele mai limitate resurse.

✓ ASA DA – MOBILE-FIRST CU MIN-WIDTH

```
.nav { display: none; }
.nav-toggle { display: flex; }
@media (min-width: 768px) {
  .nav { display: flex; }
  .nav-toggle { display: none; }
}
```

Beneficiu. Mobilul (>60% din trafic) primeste varianta optimizata implicit; desktopul adauga incremental.

9. Suport pentru tema sistem (light / dark)

Functia `light-dark()` si proprietatea `color-scheme` permit afisarea conditionata a culorilor in functie de preferinta sistemului de operare al utilizatorului:

```
1 html { color-scheme: light dark; }
2
3 :root {
4   --bg: light-dark(#FFFCF6, #0F172A);
```

```

5  --text: light-dark(#1A1A1A, #F1F5F9);
6  }

```

Browserul selectează prima sau a doua valoare în funcție de tema activă. Suportul nativ în browserele moderne este disponibil începând cu 2024 (Chrome 123+, Firefox 120+, Safari 17.5+).

10. Poziționarea elementelor

Proprietatea `position` controlează modul în care un element se așază în fluxul paginii. Cinci valori principale:

- `static` – valoarea implicită; elementul respectă fluxul natural.
- `relative` – poziție raportată la poziția normală; `top`, `left` muta vizual fără să afecteze celelalte elemente.
- `absolute` – scoate elementul din flux; poziționat relativ la primul părinte cu `position: relative` (sau viewport).
- `fixed` – poziționat relativ la viewport; rămâne vizibil la scroll. Utilizat pentru header-e sticky în stil vechi.
- `sticky` – se comportă `relative` până ajunge la o margine, apoi devine `fixed`. Modern, util pentru header-e și sidebar-uri.

Listing 10: Header sticky modern

```

1  header {
2    position: sticky;
3    top: 0;
4    background: var(--bg);
5    z-index: 10;
6  }

```

11. Animații CSS

Animațiile îmbunătățesc percepția calității produsului și orientează atenția utilizatorului. CSS suportă nativ două mecanisme: *transitions* pentru schimbări între stări, și *keyframes* pentru animații arbitrare.

11.1 Transitions – schimbări între stări

`transition` interpolează fluent o proprietate când valoarea sa se schimbă (de exemplu, la `:hover`).

Listing 11: Buton cu hover smooth

```

1  .cta {
2    background: var(--primary);
3    color: white;
4    padding: var(--space-1) var(--space-3);
5    border-radius: var(--radius);
6    transition: background 0.2s ease, transform 0.2s ease;
7  }

```

```

8
9 .cta:hover {
10     background: var(--accent);
11     transform: translateY(-2px);
12 }

```

Sintaxa: `transition: <proprietate> <durata> <timing-function> <delay>.`

✘ ASA NU – FARA TRANSITION

```

.cta { background: var(--primary); }
.cta:hover { background: var(--accent); }

```

Problema. Schimbarea este instantanee, abrupta. Utilizatorul percepe interfața ca pe un schimb de pagini, nu ca pe un raspuns.

✔ ASA DA – CU TRANSITION DE 200MS

```

.cta { background: var(--primary); transition: background 0.2s
    ease; }
.cta:hover { background: var(--accent); }

```

Beneficiu. Schimbarea este progresiva. Senzația de raspuns viu, profesional.

11.2 Transform – scalare, rotație, translație

`transform` aplica o transformare geometrica fara a afecta layout-ul:

- `transform: scale(1.05)` – marire 5%.
- `transform: rotate(45deg)` – rotire 45 grade.
- `transform: translateY(-2px)` – mutare in sus 2 pixeli.
- Combinare: `transform: scale(1.05) rotate(2deg)`.

11.3 Keyframes – animatii arbitrare

Pentru animații care nu sunt declanșate de o stare (loading spinner, fade-in, parallax), `@keyframes` definește o secvență de stări:

Listing 12: Spinner infinit

```

1 @keyframes rotate {
2     from { transform: rotate(0deg); }
3     to   { transform: rotate(360deg); }
4 }
5
6 .spinner {
7     width: 32px;
8     height: 32px;
9     border: 4px solid #E5E7EB;
10    border-top-color: var(--primary);
11    border-radius: 50%;

```

```

12   animation: rotate 1s linear infinite;
13 }

```

Listing 13: Fade-in pentru titlu hero

```

1 @keyframes fadeInUp {
2   from { opacity: 0; transform: translateY(20px); }
3   to   { opacity: 1; transform: translateY(0); }
4 }
5
6 .hero h1 {
7   animation: fadeInUp 0.6s ease-out;
8 }

```

Sintaxa shorthand pentru animation: `animation: <nume> <durata> <timing> <delay> <iteratii> <direction> <fill-mode>`.

11.4 Accesibilitate: prefers-reduced-motion

Unii utilizatori (afecțiuni vestibulare, dependente de atenție) au setat în OS preferința pentru animații reduse. CSS detectează această cerere:

```

1 @media (prefers-reduced-motion: reduce) {
2   *, *::before, *::after {
3     animation-duration: 0.01ms !important;
4     transition-duration: 0.01ms !important;
5   }
6 }

```

Conformitatea este o cerință WCAG 2.1 (criteriul 2.3.3) și o practică recomandată pentru orice site cu animații.

Verificare cunoștințe

☰ PROVOCARE

1. Care este specificitatea pentru selectorul `.btn.primary`? Comparativ, ce specificitate are `button#submit`?
2. De ce strategia mobile-first este preferabilă pentru contextul actual al traficului web?
3. *Spot the issue*: `.btn { background: red !important; }` repetat în trei locuri.
4. Care este diferența funcțională între `rem` și `px` pentru `font-size`?
5. Ce face `box-sizing: border-box` față de comportamentul implicit?

Recapitulare

✓ FAZA 2 – CSS

- Stiluri externe in `<link rel="stylesheet">`; nu inline.
- Specificitate: (`id`, `clasa`, `tag`); `!important` este *antipattern*.
- `box-sizing`: `border-box` aplicat global pentru calcul predictibil.
- Unitati: `rem` pentru fonturi/spatiere; `px` pentru border; `%/vw/vh` pentru layout responsiv.
- Variabilele CSS in `:root`; redefinire locala pentru teme contextuale.
- Flexbox: 6 proprietati cheie (`display`, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex: 1`).
- Mobile-first cu `@media (min-width: ...)`, nu desktop-first cu `max-width`.
- Pozitionare: `static` (default), `relative`, `absolute`, `fixed`, `sticky`.
- Animații: `transition` pentru hover, `@keyframes` pentru animatii arbitrare.
- Accesibilitate: respecta `prefers-reduced-motion`.



🛠️ APLICAȚIE PRACTICĂ

Platforma: Flexbox Froggy (<https://flexboxfroggy.com>).

Activitatea. Aplicatia este un set de 24 de provocari care necesita pozitionarea unor obiecte pe o grila prin folosirea proprietatilor `justify-content`, `align-items`, `flex-direction`, `order`, `flex-wrap`.

Obiectiv: primele 6 niveluri (acopera `justify-content` si `align-items`, cele mai folosite proprietati flexbox in practica).

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Mozilla Developer Network. *CSS reference*. <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Google. *Learn CSS*. <https://web.dev/learn/css>
- Coyier C. *A Complete Guide to Flexbox*. CSS-Tricks. <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- Coyier C. *A Complete Guide to Grid*. CSS-Tricks. <https://css-tricks.com/snippets/css/complete-guide-grid/>
- The Odin Project. *CSS Foundations*. <https://www.theodinproject.com/paths/foundations/courses/foundations#css-foundations>
- Aplicatii similare: <https://flukeout.github.io> (selectori), <https://cssgridgarden.com> (grid), <https://cssbattle.dev> (speedrun).

→ **Urmeaza:** Capitolul 3 (JavaScript) adauga interactivitate. Pagina stilizata in Faza 2 va raspunde la

click-uri, va schimba dark mode, va procesa formulare – fara reincarcare.

Faza 3 – JavaScript

Variabile. Functii. Pattern-ul SELECT-LISTEN-CHANGE. Manipulare DOM.

JavaScript este limbajul de programare standardizat ECMAScript (ECMA-262), interpretat nativ de toate browserele moderne. Spre deosebire de HTML (structura) și CSS (aspect), JavaScript adauga **comportament**: raspunde la actiunile utilizatorului, modifica continutul paginii dinamic, comunica cu servere prin retea.

Capitolul prezinta sintaxa de baza, modelul de programare orientat-eveniment specific paginilor web statice, și interactiunea cu arborele DOM (*Document Object Model*).

🕒 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Declara variabile cu **const** și **let**, recunoscand cand fiecare se aplica.
- Aplica pattern-ul **SELECT** → **LISTEN** → **CHANGE** pentru orice interacțiune.
- Manipuleaza DOM-ul cu **querySelector**, **classList**, **addEventListener**.
- Intercepteaza submit-ul unui formular și extrage valorile cu **FormData**.
- Diagnostheaza erorile JavaScript folosind console-ul browserului.

❓ DE CE

HTML și CSS construiesc o pagina statica. JavaScript transforma pagina intr-o aplicatie: poate reactiona la click-uri, poate schimba continutul fara reincarcare, poate trimite cereri catre servere. Pattern-ul SELECT-LISTEN-CHANGE prezentat aici acopera 80% din scenariile de interactivitate și este aceeași abstracție pe care React, Vue și Svelte o ambala in API-uri declarative.

📖 PRE-RECHIZITE

Capitolul presupune intelegerea structurii HTML (tag-uri, attribute) și a selectorilor CSS (clase, id-uri). Faze 1 și 2.

1. Conectarea fisierului JavaScript la HTML

Includerea unui fisier JavaScript se realizeaza prin elementul `<script>`, plasat in `<head>` cu atributul **defer**:

```
<script src="script.js" defer></script>
```

1.1 defer vs async vs nimic

- **Fara atribut**. Browserul opreste parsarea HTML-ului, descarca si ruleaza scriptul, apoi continua. Blocheaza randarea – antipattern.
- **async** – descarcare in paralel cu parsarea, dar executie imediat ce scriptul este disponibil (poate intrerupe parsarea). Util doar pentru scripturi independente (analytics).
- **defer** – descarcare in paralel, executie **dupa** parsarea HTML-ului. Garantat executat in

ordinea declarării. Opțiunea recomandată pentru codul propriu.

✘ ASA NU – SCRIPT IN <HEAD> FARA ATRIBUT

```
<script src="script.js"></script>
```

Problema. Când scriptul rulează, `document.querySelector('.cta')` returnează `null`, fiindcă `<body>` nu a fost încă parsat.

✔ ASA DA

```
<script src="script.js" defer></script>
```

Beneficiu. La momentul execuției, întregul DOM există. Scriptul găsește elementele cautate.

2. Declararea variabilelor

ECMAScript 2015 introduce două cuvinte cheie pentru declararea variabilelor: `const` (valoare imuabilă) și `let` (valoare mutabilă). Cuvântul cheie `var` este considerat depășit din cauza regulilor de scope.

Listing 14: Reguli de declarare

```
1 const PI = 3.14;           // valoare imutabila
2 let counter = 0;          // valoare mutabila
3
4 counter = counter + 1;     // permis
5 // PI = 3.15;             // TypeError la rulare
```

2.1 Diferența dintre var și let / const

`var` are *function scope*: o variabilă declarată în interiorul unui `if` sau `for` este accesibilă în tot blocul funcției. `let` și `const` au *block scope*: vizibile doar în interiorul `{...}` în care au fost declarate.

✘ ASA NU – VAR, SCOPE CONFUZ

```
function exemplu() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 -- scapa din if!
}
```

✔ ASA DA – LET, SCOPE PREVIZIBIL

```
function exemplu() {
  if (true) {
    let x = 10;
  }
  console.log(x); // ReferenceError -- nu exista in afara
                  if-ului
}
```

Conventia practica: `const` este valoarea implicita; `let` se foloseste doar cand reassignarea este necesara; `var` se evita complet in cod nou.

3. Tipuri primitive si compuse

```

1  const nume      = "Maria";           // string
2  const varsta    = 21;                 // number (intregi si
    zecimale)
3  const eStudent  = true;              // boolean
4  const limbaje   = ["HTML", "CSS"];   // array (lista ordonata)
5  const persoana  = {                  // object (pereche
    cheie-valoare)
6    nume: "Maria",
7    varsta: 21,
8  };
9
10 console.log(persoana.nume);           // "Maria"
11 console.log(limbaje[0]);              // "HTML"
12 console.log(typeof varsta);           // "number"

```

JavaScript este un limbaj cu *tipare dinamica*: tipul unei variabile este determinat de valoarea atribuita si poate fi modificat in timpul executiei.

3.1 Verificarea egalitatii: `===` vs `==`

Operatorul `==` efectueaza *coercitie de tip*: converteste operanzii inainte de comparatie. Rezultatul este des contraintuitiv. Operatorul `===` compara strict valoare si tip.

✘ ASA NU – `==`, COERCITIE SURPRINZATOARE

```

0 == "0"           // true
0 == ""            // true
"" == false        // true
null == undefined  // true
[] == false        // true

```

✔ ASA DA – `===` MEREU

```

0 === "0"          // false (number vs string)
0 === ""           // false
null === undefined // false

```

4. Functii: declaratie, expresie, arrow

JavaScript ofera trei sintaxe pentru definirea unei functii. Cele trei comportamente difera la nivel de `this` si hoisting, dar pentru cazul uzual (handler de eveniment) sunt interschimbabile.

Listing 15: Trei moduri de a scrie aceeași funcție

```

1 // 1. Declarație de funcție (function declaration)
2 function saluta(ume) {
3     return `Salut, ${ume}!`;
4 }
5
6 // 2. Expresie de funcție (function expression)
7 const saluta2 = function(ume) {
8     return `Salut, ${ume}!`;
9 };
10
11 // 3. Arrow function (sintaxa concisă)
12 const saluta3 = (ume) => `Salut, ${ume}!`;
13
14 // Apel: identic în toate cazurile
15 console.log(saluta("Maria"));

```

Convenția practică: arrow functions pentru handlers și callbacks; declarații pentru funcții cu nume reutilizate, definite la nivelul fișierului.

5. Pattern-ul SELECT → LISTEN → CHANGE

Majoritatea interacțiunilor pe paginile statice respectă o schemă în trei pași: selectarea elementului DOM, atasarea unui ascultător de eveniment, modificarea elementului în răspuns la eveniment.

Listing 16: Toggle pentru tema dark/light

```

1 // 1. SELECT
2 const btn = document.querySelector('.toggle-theme');
3
4 // 2. LISTEN
5 btn.addEventListener('click', () => {
6
7     // 3. CHANGE
8     document.body.classList.toggle('dark');
9
10 });

```

🔗 PONT

Pattern-ul descris reprezintă abstractizarea fundamentală pe care framework-urile moderne (React, Vue, Svelte, Angular) o ascund în spatele propriilor API-uri declarative. Înțelegerea variantei *vanilla* facilitează tranziția ulterioară către framework-uri.

6. Manipularea DOM-ului

API-ul `document` expune metode pentru selecția și modificarea nodurilor:

Listing 17: Operatii uzuale

```

1 // SELECT -- un singur element
2 const el = document.querySelector('.cta'); // primul .cta
3 const id = document.getElementById('contact'); // alternativa
  pt id
4
5 // SELECT -- mai multe elemente
6 const linkuri = document.querySelectorAll('nav a');
7 linkuri.forEach((link) => {
8   link.addEventListener('click', () => console.log('click'));
9 });
10
11 // CHANGE -- continut
12 el.textContent = 'Salut!'; // text simplu (sigur)
13 el.innerHTML = '<strong>Salut!</strong>'; // HTML (atentie XSS)
14
15 // CHANGE -- attribute
16 el.setAttribute('aria-expanded', 'true');
17 el.classList.add('active');
18 el.classList.remove('hidden');
19 el.classList.toggle('open');
20
21 // CHANGE -- creare element nou
22 const nou = document.createElement('li');
23 nou.textContent = 'Element nou';
24 document.querySelector('ul').appendChild(nou);

```

✖ ASA NU – QUERYSELECTOR LA FIECARE APEL

```

btn.addEventListener('click', () => {
  document.querySelector('.modal').classList.add('open');
  document.querySelector('.modal').focus();
  document.querySelector('.modal').scrollIntoView();
});

```

Problema. Trei traversari ale DOM-ului pentru aceeași referință.

✔ ASA DA – CACHE LOCAL

```

const modal = document.querySelector('.modal');

btn.addEventListener('click', () => {
  modal.classList.add('open');
  modal.focus();
  modal.scrollIntoView();
});

```

7. Procesarea formularelor

Submit-ul nativ al unui formular trimite o cerere HTTP catre URL-ul declarat in `action` si reincarca pagina. In majoritatea cazurilor, fluxul dorit este interceptarea evenimentului si manipularea datelor in JavaScript:

Listing 18: Captura datelor de formular fara reincarcarea paginii

```
1 const form = document.querySelector('#contact-form');
2
3 form.addEventListener('submit', (e) => {
4   e.preventDefault(); // blocheaza
      reincarcarea
5
6   const data = Object.fromEntries(new FormData(form)); // {nume,
      email, mesaj}
7   console.log('Date primite:', data);
8
9   form.reset(); // goleste
      campurile
10  alert('Multumesc, ${data.nume}!');
11 });
```

Fluxul: `e.preventDefault()` suspenda comportamentul implicit; `new FormData(form)` colecteaza valorile dupa atributul `name`; `Object.fromEntries(...)` le converteste intr-un obiect uzual; `form.reset()` reseteaza valorile in interfata.

🔗 PONT

Pentru transmiterea efectiva a datelor catre un endpoint extern (fara backend propriu), serviciul [Formspree](#) ofera un endpoint gratuit. Configurarea consta intr-o singura modificare a atributului `action`.

8. Debugging si console

Console-ul browserului (F12 → tab *Console*) este principalul instrument de diagnostic. Erorile JavaScript apar automat aici cu mesaj, fisier si linie.

Listing 19: Trei moduri de a inspecta valorile

```
1 // Cel mai uzual: log valoare cu eticheta
2 console.log('Date primite:', data);
3
4 // Tabel pentru obiecte / array-uri (formatare imbunatatita)
5 console.table(linkuri);
6
7 // Pauza in executie -- DevTools deschis
8 debugger;
```

Sfaturi practice. Inainte de a cere ajutor, citeste mesajul de eroare din console. Numele functiei si numarul liniei sunt aproape intotdeauna corecte. `console.log(typeof x)` verifica tipul cand

comportamentul este surprinzător.

9. Tratarea erorilor (try/catch)

Codul care poate genera erori la rulare (parsare JSON, apeluri `fetch`, acces la API-uri externe) trebuie încadrat într-un bloc `try/catch` pentru ca o eroare să nu oprească executia restului paginii:

Listing 20: Parsare JSON cu fallback

```
1 const raw = localStorage.getItem('preferinte');
2
3 try {
4   const config = JSON.parse(raw);
5   console.log('Preferinte incarcate:', config);
6 } catch (err) {
7   console.warn('JSON invalid in localStorage; folosesc valori
      implicite');
8   // continuare cu valori default
9 }
```

Când eroarea este previzibilă și recuperabilă (utilizatorul poate continua), `try/catch` este abordarea corectă. Pentru erori neprevăzute, evenimentul global `window.addEventListener('error', ...)` permite logarea către un serviciu de monitoring.

10. Erori frecvente

- `Cannot read property '...' of null`. Cauza: `querySelector` a returnat `null` și codul a încercat să acceseze o proprietate. Diagnostic: `console.log(el)` înainte de a folosi.
- `addEventListener is not a function`. Aceeași cauză: rezultatul `querySelector` este `null`.
- **Hoisting**. Variabilele `let` și `const` declarate într-un bloc nu sunt accesibile înainte de declarație. Accesul anticipat generează *ReferenceError: Cannot access before initialization*.
- **Comparatie cu `undefined`**. Folosește `x === undefined`, nu `!x` – ultimul prinde și `0`, `""`, `false`, `null`.

Verificare cunoștințe

☰ PROVOCARE

1. Ce face atributul `defer` pe un `<script>` și de ce este necesar?
2. Care sunt cei trei pași ai pattern-ului **SELECT** → **LISTEN** → **CHANGE**?
3. De ce `===` este preferabil lui `==`? Dați un exemplu de coerciție surprinzătoare.
4. Ce face `e.preventDefault()` apelat într-un handler de `submit`?
5. Spot the issue: `const btn = document.querySelector('.cta');`
`btn.addEventListener(...);` – `btn` este `null`.

Recapitulare

✓ FAZA 3 – JAVASCRIPT

- Includere: `<script src="..." defer></script>` in `<head>`.
- `const` este declararea implicita; `let` doar cand reassignarea este necesara; nu `var`.
- Comparatie: exclusiv `===`, niciodata `==`.
- Pattern-cheie: **SELECT** → **LISTEN** → **CHANGE** acopera majoritatea interactiunilor.
- DOM: `querySelector` (un element); `querySelectorAll` (NodeList iterabil cu `.forEach`).
- Form submit: `e.preventDefault()` + `new FormData(form) + Object.fromEntries(...)`.
- Cache referintele DOM in variabile; nu repeta `querySelector` pentru acelasi element.
- Debug: `F12` → `Console`, `console.log()`, `debugger`.



🔧 APLICAȚIE PRACTICĂ

Platforma: CodePen (codepen.io). URL-ul exact este distribuit de trainer in chat.

Activitatea. Documentul de pornire contine HTML si CSS complet, dar fisierul JavaScript este gol. Sarcina este implementarea pattern-ului SELECT-LISTEN-CHANGE: la apasarea butonului prezent in pagina, clasa `dark` se aplica si se scoate alternativ pe elementul `<body>`.

Solutia asteptata:

```
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
});
```

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Kantor I. *The Modern JavaScript Tutorial*. <https://javascript.info>
- Mozilla Developer Network. *JavaScript reference*.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- The Odin Project. *JavaScript Foundations*.
<https://www.theodinproject.com/paths/foundations/courses/foundations#javascript-basics>
- Mozilla Developer Network. *Document Object Model*.
https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- Google. *Learn JavaScript*. <https://web.dev/learn/javascript>

→ **Urmeaza:** Capitolul 4 (Deploy) muta site-ul de pe mașina locala pe un URL public. Toate fazele 1-3 sunt locale; abia dupa deploy pagina exista in lume.

Faza 4 – Deploy

Versionare cu git. Publicare prin GitHub Pages.

Capitolul descrie procesul de publicare a unui site web static prin combinația git – GitHub – GitHub Pages. Acoperirea include comenzile esențiale ale fluxului de lucru, configurarea unui repository remote, activarea serviciului de hosting integrat, și convenii de scriere a mesajelor de commit.

🕒 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Executa cele 5 comenzi git pentru inițializarea unui repository și primul push.
- Configurează GitHub Pages pe un repository pentru hosting static automat.
- Folosește `git status`, `git diff`, `git log` pentru inspectarea stării.
- Scrie mesaje de commit la imperativ, descriptive, sub 72 caractere.
- Excludere fișiere sensibile prin `.gitignore`.

❓ DE CE

Site-ul construit local pe mașina dezvoltatorului nu este accesibil nimănui altcuiva. Deploy-ul transformă cod local în URL public. În plus, git este standardul industrial pentru versionare – cunoștințele dobândite aici se aplică identic în orice job de dezvoltare software. Mesajele de commit bune sunt diferența între un istoric care ajută la debug și unul inutil.

📖 PRE-RECHIZITE

Capitolul presupune un site funcțional în localhost (Faza 1–3) și un cont GitHub creat în prealabil.

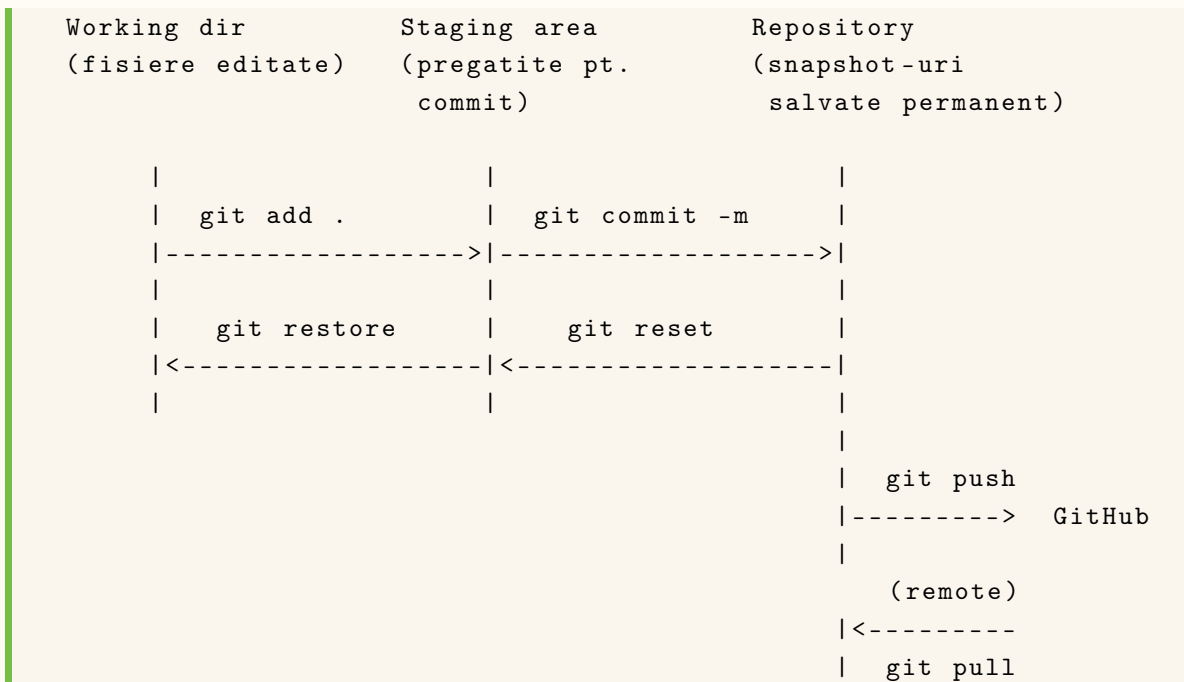
1. De ce git?

Git este sistemul de control al versiunilor (*Version Control System*) folosit de aproape toată industria software. Beneficiile față de o soluție de tip „upload manual prin FTP”:

- **Istoric.** Fiecare commit reprezintă un *snapshot* al codului, identificat printr-un hash unic. Reverența la o versiune anterioară este o operație atomică.
- **Backup distribuit.** Repository-ul există local, pe GitHub, și pe orice copie clonată. Pierderea unei singure copii nu afectează istoricul.
- **Colaborare.** Mai mulți autori pot lucra simultan pe ramuri (*branches*) separate, sincronizate prin merge.
- **Standard industrial.** Cunoașterea git este o cerință de bază în posturile de dezvoltare software.

2. Modelul mental: trei zone

Git menține trei zone în care fișierele pot exista la un moment dat:



- **Working directory** – fișierele așa cum apar pe disc.
- **Staging area** (sau *index*) – multimea fișierelor pregătite pentru următorul commit.
- **Repository** – istoricul snapshot-urilor confirmate.

Comenzile `add`, `commit`, `push` muta starea între zone în direcția stanga → dreapta.

3. Cele cinci comenzi de baza

Fluxul minim pentru publicarea unui director nou pe GitHub:

```

1 git init -b main # 1
2 git add . # 2
3 git commit -m "Initial commit" # 3
4 git remote add origin URL_DE_PE_GITHUB # 4
5 git push -u origin main # 5

```

- `git init -b main` – initializează un repository local cu ramura implicită `main`.
- `git add .` – include toate fișierele modificate în zona de pregătire (*staging area*).
- `git commit -m "..."` – creează un snapshot al stării pregătite, cu un mesaj descriptiv.
- `git remote add origin ...` – asociază repository-ul local cu un remote pe GitHub.
- `git push -u origin main` – trimite commit-urile locale către remote, stabilind ramura `main` ca *upstream*.

4. Comenzi de inspectie (read-only)

Înainte de `commit`, două comenzi confirmă ce urmează să fie salvat:

Listing 21: Verificare inainte de commit

```

1 git status          # Lista fișierelor modificate / pregătite
2 git diff            # Diferența linie cu linie (working dir)
3 git diff --staged    # Diferența în staging area
4 git log --oneline -10 # Ultimele 10 commit-uri, format compact

```

Aceste comenzi nu modifică nimic; pot fi rulate oricând fără risc.

5. Configurarea GitHub Pages

Procedura standard pentru activarea hosting-ului:

1. Creare a unui repository public la <https://github.com/new>, fără fișiere implicite (README, gitignore).
2. Copierea URL-ului HTTPS afișat de GitHub și executarea comenzii `git remote add origin <URL>` urmată de `git push -u origin main`.
3. Navigare la *Settings* → *Pages* (în panoul lateral al repository-ului).
4. Selectarea ramurii `main` și a folder-ului `/ (root)` în secțiunea *Branch*, urmată de *Save*.
5. Așteptarea de 30 – 60 secunde, după care URL-ul public devine accesibil în caseta verde din partea de sus.

Anatomia URL-ului public: <https://USERNAME.github.io/REPO>

6. Workflow zilnic după primul push

Actualizarea conținutului după deploy-ul inițial necesită doar trei comenzi:

```

1 git status          # ce s-a modificat?
2 git add .           # marcarea pentru commit
3 git commit -m "Adauga secțiunea proiecte"
4 git push            # trimitere pe GitHub

```

GitHub Pages reconstruiește și republică pagina automat, în aproximativ 60 de secunde.

7. Convenii pentru mesajele de commit

Mesajul de commit rămâne în istoricul repository-ului permanent. Calitatea acestuia este vizibilă la `git log` și afectează utilitatea istoricului ca documentație a evoluției proiectului.

✖ ASA NU

```

git commit -m "update"
git commit -m "fix"
git commit -m "asdf"
git commit -m "Modificari"
git commit -m "."

```

Probleme. Niciun context. La revizuire, autorul nu mai știe ce conține fiecare commit. Inutilizabil

pentru `git revert`, `git bisect`, code review.

✓ ASA DA

```
git commit -m "Adauga sectiunea hero cu CTA"
git commit -m "Fix overflow horizontal pe mobil < 360px"
git commit -m "Implementeaza dark mode prin light-dark()"
git commit -m "Reduce dimensiunea hero.jpg de la 2.4MB la 180KB"
```

Reguli simple. (1) Verb la imperativ in prima pozitie (*Adauga*, *Fix*, *Implementeaza*, *Reduce*). (2) Maxim 50–72 caractere. (3) Descrie *ce si de ce*, nu *cum* (codul arata cum).

8. Fisierul `.gitignore`

Anumite fisiere si directoare nu trebuie sa ajunga in repository: dependinte instalate (`node_modules/`), fisiere temporare ale editoarelor, secretele (`.env`). Lista lor se declara in fisierul `.gitignore` la radacina proiectului:

Listing 22: Exemplu minimal pentru proiect web static

```
1 # Dependinte
2 node_modules/
3
4 # Editoare
5 .vscode/
6 .idea/
7 *.swp
8
9 # Sistem
10 .DS_Store
11 Thumbs.db
12
13 # Secrete
14 .env
15 .env.local
16
17 # Build output
18 dist/
19 build/
```

Fisierul se commit-eaza ca orice alt fisier. Toate caile listate sunt ignorate la `git add`.

⚠ ATENȚIE

Nu commite niciodata fisiere `.env` cu chei API, parole, token-uri. Odata commit-ate, raman in istoric chiar dupa stergere; pot fi recuperate de oricine cu acces la repository. Daca s-a intamplat: regenerati cheia compromisa imediat la furnizorul respectiv.

9. Probleme frecvente

⚠ ATENȚIE

Autentificare prin parola. GitHub a renunțat la autentificarea prin parola pentru operațiunile git (din 2021). Soluția recomandată este generarea unui *Personal Access Token* (PAT) la <https://github.com/settings/tokens> cu scope-ul `repo`, folosit ulterior în locul parolei. Browserul memorează tokenul pentru sesiunile viitoare.

⚠ ATENȚIE

Eroare 404 pe pagina deployata. Cauze posibile: (1) intervalul de propagare nu a fost respectat (60 secunde după *Save*); (2) fișierul de pornire nu se numește `index.html` (este sensibil la majuscule pe serverul GitHub); (3) cache-ul browserului afișează versiunea precedentă – se rezolvă cu `Ctrl+F5`.

Verificare cunoștințe

≡ PROVOCARE

1. Care sunt cele 5 comenzi git pentru inițializare și primul push?
2. De ce GitHub a renunțat la autentificarea prin parola? Care este alternativa?
3. *Spot the issue:* `git commit -m "update"`.
4. Diferența între `git status` și `git diff`?
5. Pentru un repository numit `portfolio` al utilizatorului `maria`, care este URL-ul GitHub Pages rezultat?

Recapitulare

✔ FAZA 4 – DEPLOY

- Modelul mental: `working dir` → `staging` → `repository` → `remote`.
- Cinci comenzi de baza: `git init -b main`, `git add .`, `git commit -m`, `git remote add origin`, `git push -u origin main`.
- Comenzi de inspectie: `git status`, `git diff`, `git log -oneline -10`.
- Activare Pages: `Settings` → `Pages` → branch `main`, folder / (root) → `Save`.
- Mesaje de commit: verb la imperativ, ≤72 caractere, descrie *ce* și *de ce*.
- Workflow zilnic: 3 comenzi (`add`, `commit`, `push`); redeploy automat în ~60s.
- Autentificare prin Personal Access Token (PAT), nu prin parola.
- `.gitignore` exclude `node_modules/`, `.env`, fișiere de editor, build artifacts.

• • •

🔗 APLICAȚIE PRACTICĂ

Platforma: GitHub (<https://github.com>) – aplicația practică este deploy-ul efectiv al site-ului construit în fazele 1–3.

Activitatea. Procedura completă, executată sincron de toți participanții sub îndrumarea trainer-ului:

1. Inchiderea fisierelor in editor; navigare in *Git Bash* la directorul proiectului.
2. Executarea celor cinci comenzi git din sectiunea „Cele cinci comenzi de baza”.
3. Crearea repository-ului public pe GitHub si conectarea remote-ului.
4. Activarea GitHub Pages in setari.
5. Verificarea URL-ului public si publicarea acestuia in canalul Discord al grupului.

Durata estimata: 10 minute.

Resurse pentru aprofundare

- GitHub Inc. *GitHub Pages documentation*. <https://pages.github.com>
- Cottle P. *Learn Git Branching*. <https://learngitbranching.js.org>
- Chacon S., Straub B. *Pro Git* (editia a 2-a). <https://git-scm.com/book/en/v2>
- GitHub Inc. *Configuring a custom domain for your GitHub Pages site*.
<https://docs.github.com/pages/configuring-a-custom-domain-for-your-github-pages-site>
- Beams C. *How to Write a Git Commit Message*. <https://cbea.ms/git-commit/>
- Alternative: <https://www.netlify.com>, <https://vercel.com> (servicii similare cu deploy automat din git).

→ **Urmeaza:** Capitolul 5 (SEO) face site-ul descoperibil. URL public exista; acum trebuie ca motoarele de cautare și rețelele sociale sa-l afișeze corect.

Faza 5 – SEO + Open Graph

Metadate pentru motoarele de cautare si rețelele sociale.

Capitolul prezinta tag-urile meta utilizate de motoarele de cautare (in principal Google) si de platformele sociale (Facebook, X, LinkedIn, WhatsApp, Discord) pentru afisarea corecta a paginilor distribuite. Acoperirea include SEO clasic, protocolul Open Graph (OGP), datele structurate Schema.org, si URL-urile canonice.

🎯 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Configureaza `<title>` și `<meta description>` pentru rezultatele de cautare.
- Adauga cele 5 tag-uri `og:*` pentru previzualizare in rețele sociale.
- Genereaza și valideaza imaginea `og:image` la dimensiunile și formatul corect.
- Adauga date structurate (JSON-LD) pentru rich snippets in Google.
- Distinge mit-urile SEO de practica actuala.

❓ DE CE

Un site fara SEO este invizibil. Aplicațiile sociale moderne (WhatsApp, Discord, LinkedIn) afișează by default cardul de previzualizare; un site fara meta tags arata neprofesional cand cineva il share-uieste. SEO de baza este *configurare*, nu *magic*: cele 7-10 tag-uri prezentate aici acopera 80% din optimizarea esentiala.

📌 PRE-RECHIZITE

Capitolul presupune un site deployat (Faza 4) cu URL public accesibil. Tag-urile se valideaza pe varianta live, nu pe localhost.

1. Doua audiente, doua seturi de tag-uri

- **Motoare de cautare** (Google, Bing). Folosesc `<title>`, `<meta name="description">`, structura semantica, datele Schema.org.
- **Platforme sociale** (Facebook, X, LinkedIn, WhatsApp, Discord). Folosesc protocolul Open Graph: `<meta property="og:*">`.

2. Metadate pentru motoarele de cautare

Doua elemente determina aspectul paginii in rezultatele cautarii:

Listing 23: Plasare in `<head>`

```
1 <title>Maria Popescu -- Student CS, Iasi</title>
2 <meta name="description" content="Pagina personala a Mariei Popescu,
  studenta la CS, pasionata de web development.">
```


2.1 Calitatea <title>

✖ ASA NU

```
<title>Document</title>
<title>Untitled</title>
<title>Index</title>
<title>Acasa</title>
<title>web web web html portofoliu cv student iasi cs
    uaic</title>
```

Probleme. Generic, nesemnificativ; sau spam de cuvinte-cheie penalizat de Google.

✔ ASA DA

```
<title>Maria Popescu -- Portofoliu Web Developer, Iasi</title>
<title>Despre BEST Iasi -- Asociatie studenteasca tehnica</title>
```

Reguli. 50–60 caractere; specific paginii (nu site-ului); structura „Subiect – Context”.

2.2 Calitatea <meta description>

✖ ASA NU

```
<meta name="description" content="Pagina mea">
<meta name="description" content="Bine ati venit pe site-ul meu! Aici puteti
vedea cele mai noi informatii despre activitatea mea! Va invit sa exploratati!
Multumesc!">
```

Probleme. Prea scurt sau plin de fraze de umplutura.

✔ ASA DA

```
<meta name="description" content="Portofoliul Mariei Popescu: proiecte web (HTML,
CSS, JavaScript, React), CV, contact. Studenta in anul III la UAIC.">
```

Reguli. 150–160 caractere; expresii cheie (proiecte, CV, contact); descriere a continutului concret.

3. Protocolul Open Graph

Open Graph Protocol (OGP), specificat de Facebook in 2010, este standardul de fapt pentru previzualizarea paginilor web in retelele sociale. Implementarea consta in adaugarea a cinci tag-uri <meta> cu prefixul og::

Listing 24: Tag-uri OGP minimale

```
1 <meta property="og:type" content="website">
2 <meta property="og:title" content="Maria Popescu -- Student CS,
   Iasi">
3 <meta property="og:description" content="Pagina personala. Web dev,
   design, open source.">
4 <meta property="og:image"
   content="https://maria.github.io/site/og.png">
5 <meta property="og:url" content="https://maria.github.io/site/">
```

- `og:type` – categoria conținutului. Valoarea `website` acopera majoritatea cazurilor; alternative: `article`, `video`, `book`, `profile`.
- `og:title` și `og:description` – pot diferi de `<title>` și `<meta description>`, fiind optimizate specific pentru context social (mai punctuale, cu CTA).
- `og:image` – dimensiunea recomandată 1200 x 630 pixeli (raport 1.91:1).
- `og:url` – URL-ul canonic al paginii.

3.1 Constrângerea `og:image`: URL absolut

✗ ASA NU – URL RELATIV

```
<meta property="og:image" content="og.png">
<meta property="og:image" content="/images/og.png">
```

Problema. Crawler-ele platformelor sociale nu pot rezolva caile relative, întrucât nu cunosc contextul de bază al paginii sursă. Cardul afișat rămâne fără imagine.

✓ ASA DA – URL ABSOLUT

```
<meta property="og:image" content="https://maria.github.io/site/og.png">
```

Beneficiu. Imaginea este accesibilă direct prin URL și poate fi cache-uită de Facebook, Slack, Discord.

3.2 Calitatea imaginii `og:image`

- Dimensiune ideală: **1200 x 630 px** (raport 1.91:1, formatul afișat de Facebook și LinkedIn).
- Dimensiune minimă: 600 x 315 px. Sub această limită, unele platforme nu afișează cardul mare.
- Format: JPG sau PNG; dimensiunea maximă 8 MB (Facebook), în practică ≤ 1 MB pentru încărcare rapidă.
- Conținut lizibil la dimensiune redusă: titlul + 1–2 elemente vizuale, nu paragrafe întregi.

4. Twitter Card

Platforma X (anterior Twitter) suportă nativ OGP, dar acceptă și tag-uri proprii pentru carduri cu imagine mare:

```
<meta name="twitter:card" content="summary_large_image">
```

Adăugarea acestei singure linii instruieste X să afișeze cardul în formatul extins, folosind valorile deja prezente în tag-urile `og:*`.

5. Date structurate (JSON-LD)

Schema.org definește un vocabular standardizat pentru descrierea entităților (persoane, organizații, evenimente, articole) într-un format pe care motoarele de căutare îl procesează explicit. Implementarea recomandată: JSON-LD plasat într-un `<script>` cu tipul `application/ld+json`.

Listing 25: Schema Person pentru o pagina personala

```

1 <script type="application/ld+json">
2 {
3   "@context": "https://schema.org",
4   "@type": "Person",
5   "name": "Maria Popescu",
6   "url": "https://maria.github.io/site/",
7   "jobTitle": "Student CS",
8   "alumniOf": "Universitatea Alexandru Ioan Cuza",
9   "sameAs": [
10    "https://github.com/maria",
11    "https://www.linkedin.com/in/maria"
12  ]
13 }
14 </script>

```

Avantajul principal: Google poate afisa pagina cu *rich snippets* (foto, link-uri sociale, descriere structurata) in rezultatele cautarii. Validarea se face cu Google Rich Results Test (<https://search.google.com/test/rich-results>).

🔗 PONT

Pentru un eveniment, foloseste "@type": "Event" cu campurile `startDate`, `location`, `organizer`. Google poate afisa evenimentul direct intr-un panel lateral. Vocabularul complet: <https://schema.org>.

6. URL-ul canonic

Cand acelasi continut este accesibil prin mai multe URL-uri (cu si fara `www`, cu si fara slash final, cu parametri UTM, etc.), Google trebuie sa stie care URL este versiunea „oficiala”. Tag-ul canonical previne penalizarea pentru continut duplicat:

```

1 <link rel="canonical" href="https://maria.github.io/site/">

```

Indicatie: pe paginile de proiect, URL-ul canonic este URL-ul „curat” al paginii, fara parametri de tracking.

7. Mituri SEO

- **Mit:** `<meta name="keywords">` influenteaza ranking-ul.
- **Realitate:** Google a anuntat oficial in 2009 ca ignora complet acest tag. Bing si Yandex il folosesc minim. **A nu se mai folosi.**
- **Mit:** repetarea cuvintelor cheie in continut creste ranking-ul.
- **Realitate:** *keyword stuffing* este penalizat. Algoritmi modernii (BERT, MUM) inteleg sensul textului si premiaza scrierea naturala.
- **Mit:** site-urile noi sunt penalizate de Google.
- **Realitate:** indexarea dureaza 1–7 zile pentru un site nou. Inscrierea in Google Search Con-

sole (<https://search.google.com/search-console>) accelereaza procesul.

8. Favicon

Favicon-ul (iconita afisata in tab-ul browserului si in bookmark-uri) se declara prin elementul

`<link>`:

```
1 <link rel="icon" href="favicon.svg" type="image/svg+xml">
2 <link rel="apple-touch-icon" href="apple-touch-icon.png">
```

Generarea unui favicon simplu se poate face cu serviciul <https://favicon.io/favicon-generator/>, care produce fisierele necesare pornind de la o singura litera si o culoare. Pentru un favicon profesional din SVG, conversia automata in toate dimensiunile necesare se face cu <https://realfavicongenerator.net>.

Verificare cunoștințe

☰ PROVOCARE

1. Cate tag-uri `og:*` sunt minim necesare pentru un card valid de previzualizare?
2. De ce `og:image` trebuie sa fie URL absolut, nu relativ?
3. *Spot the issue:* `<title>Document</title>`.
4. Care este diferența funcțională între `<title>` si `og:title`?
5. De ce `<meta name="keywords">` este considerat depasit?

Recapitulare

📌 FAZA 5 – SEO + OPEN GRAPH

- Doua audiente: motoare de cautare (`<title>` + `<meta description>`) si retele sociale (5 `og:*` tags).
- `<title>`: 50–60 caractere, specific, structura „Subiect – Context”.
- `<meta description>`: 150–160 caractere, descriere concreta, fara umplutura.
- `og:image`: URL absolut, dimensiune optima 1200 x 630, format JPG/PNG, ≤1MB.
- `<meta name="twitter:card" content="summary_large_image">` activeaza cardul mare pe X.
- JSON-LD (Schema.org) descrie entitati: Person, Organization, Article, Event.
- `<link rel="canonical">` previne penalizarea pentru continut duplicat.
- `<meta name="keywords">` este ignorat de Google din 2009 – **a nu se folosi**.

• • •

🛠️ APLICAȚIE PRACTICĂ

Platforma: OpenGraph.xyz (<https://opengraph.xyz>).

Activitatea.

1. Adaugarea celor cinci tag-uri `og:*` în `<head>`, cu valori specifice paginii personale.
2. Inlocuirea valorii pentru `og:image` cu un URL absolut către o imagine 1200x630, accesibilă public.
3. Commit și push: `git add .; git commit -m "Adauga meta tags Open Graph"; git push.`
4. Accesarea OpenGraph.xyz și lipirea URL-ului public al paginii.
5. Verificarea previzualizărilor pentru Facebook, X, LinkedIn, WhatsApp, iMessage.

Diagnoza problemelor: (1) URL-ul `og:image` nu este absolut; (2) imaginea returnează HTTP 404; (3) dimensiunea imaginii este sub 600 pixeli pe oricare axă.

Durata estimată: 5 minute.

 Resurse pentru aprofundare

- Open Graph Protocol. *Specificația oficială*. <https://ogp.me>
- Meta Tags. *Generator vizual*. <https://metatags.io>
- Google. *Rich Results Test*. <https://search.google.com/test/rich-results>
- Google. *Search Console* (indexare manuală). <https://search.google.com/search-console>
- Google. *Document metadata*. <https://web.dev/learn/html/metadata>
- Schema.org. *Structured data documentation*. <https://schema.org/docs/gs.html>
- X Inc. *Twitter Cards documentation*. <https://developer.x.com/en/docs/twitter-for-websites/cards>

→ **Urmeaza:** Capitolul 6 (Lighthouse) oferă o măsurătoare obiectivă a calității. Toate fix-urile aplicate în fazele 1–5 sunt acum cuantificate într-un scor 0–100.

Faza 6 – Lighthouse

Audit automatizat. Indicatori de performanta. Core Web Vitals.

Lighthouse este un instrument de audit automatizat dezvoltat de Google, integrat in Chrome DevTools si in serviciul web PageSpeed Insights. Capitolul prezinta cele patru categorii evaluate, modul de interpretare a raportului, indicatorii *Core Web Vitals* folositi de Google in algoritmul de ranking, si remedierile pentru problemele frecvente.

🕒 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Ruleaza un audit Lighthouse pe propria pagina prin PageSpeed Insights sau DevTools.
- Citește raportul și identifica problemele cu impact maxim pe scor.
- Aplica fix-uri uzuale: `alt`, `lang`, `width/height` pe imagini, `defer` pe script.
- Optimizeaza imaginile cu format modern (WebP), `srcset`, `loading="lazy"`.
- Interpreteaza Core Web Vitals (LCP, INP, CLS) și pragurile lor.

❓ DE CE

Lighthouse este standardul industrial pentru evaluarea unei pagini web. Scorul 95+ pe toate cele 4 categorii este criteriul implicit la code review in firmele moderne. Mai mult, Core Web Vitals influenteaza direct ranking-ul Google. Un audit de 2 minute identifica 80% din problemele care ar costa ore de cautare manuala.

📄 PRE-RECHIZITE

Capitolul presupune un site deployat (Faza 4) si configurat cu meta tags (Faza 5). Auditul ruleaza pe URL-ul public.

1. Categoriile de evaluare

Lighthouse genereaza un scor 0–100 pe fiecare dintre cele patru axe:

- **Performance** – timpul de incarcare si receptivitatea paginii.
- **Accessibility** – conformitatea cu standardele WCAG (in principal contrast, structura semantica, asocieri label–input, attribute `alt`).
- **Best Practices** – folosirea de protocoale si API-uri moderne, absenta erorilor in consola, lipsa codului depasit.
- **SEO** – prezenta meta tag-urilor obligatorii, structura semantica, indexabilitatea.

Scorurile se interpreteaza astfel: peste 90 = bun; peste 95 = nivel profesional; sub 50 = necesita interventie.

1.1 Lab data vs field data

Lighthouse foloseste *lab data*: un scenariu sintetic in conditii standardizate (CPU lent, retea simulata 4G slow). Scorul reflecta capacitatea pe un dispozitiv mediu, nu situatia reala a utilizatorilor.

Pentru date din productie, PageSpeed Insights afiseaza si *field data* extrase din Chrome User Experience Report (CrUX), agregate pe ultimele 28 de zile.

2. Modalitati de executie

2.1 PageSpeed Insights (interfata web)

Cea mai accesibila metoda. Pasi: accesarea <https://pagespeed.web.dev>, lipirea URL-ului public al paginii, executarea analizei. Avantaje: nu necesita instalare; include datele reale de la utilizatori (CrUX) atunci cand acestea exista; raport partajabil prin URL.

2.2 Chrome DevTools (local)

Procedura: deschiderea paginii in Chrome, **F12** → tab *Lighthouse*, selectarea categoriilor, executarea *Analyze page load*. Avantaje: functioneaza pe paginile servite de pe `localhost`, fara necesitatea de deploy; permite testare rapida intre modificari.

2.3 CLI (pentru integrare CI)

Lighthouse este disponibil ca pachet npm. Folosit in pipeline-uri CI/CD pentru a bloca deploy-urile care scad scorul sub un prag stabilit.

3. Structura raportului

Fiecare categorie include:

- Scorul numeric, evidentiat color (rosu / portocaliu / verde).
- Lista problemelor identificate, grupate pe tip. Selectarea unei probleme afiseaza descrierea, impactul, si pasii de remediere.
- Lista verificarilor trecute cu succes.
- Sectiunea *Opportunities* (pentru Performance) listeaza imbunatatirile cu economia estimata in milisecunde.

4. Probleme frecvente si remediere

4.1 Performance

- **Imagini fara dimensiuni explicite.** Setarea atributelor `width` si `height` pe `<img>` permite browserului sa rezerve spatiu, prevenind *Cumulative Layout Shift*.
- **Resurse care blocheaza randarea.** Adaugarea atributului `defer` pe `<script>` permite parsarea HTML in paralel.
- **Imagini supradimensionate.** O imagine 2400x1600 redactata la 800 pixeli iroseste banda. Optimizare: <https://squoosh.app>.
- **Format imagini neoptimizat.** JPEG/PNG in locul WebP / AVIF.
- **Cache lipsa.** Resursele statice (CSS, JS, imagini) ar trebui servite cu header `Cache-Control: max-age=31536000` (1 an). Pe GitHub Pages: cache-ul este configurat automat, fara interventie.

4.2 Accessibility

- **Atribut alt lipsa pe .** Toate imaginile relevante semantic necesita `alt`; imaginile pur decorative folosesc `alt=""`.
- **Buton fara nume accesibil.** Includerea de text in interior sau adaugarea `aria-label`.
- **Etichete lipsa pe formulare.** Asocierea `<label for>` cu `<input id>`.
- **Contrast insuficient.** Conform WCAG AA, raportul de contrast minim este 4.5:1 pentru text normal si 3:1 pentru text mare. Verificare: <https://webaim.org/resources/contrastchecker/>.

4.3 Best Practices

- **Erori in consola.** Lighthouse penalizeaza orice eroare `console.error` la incarcare. Verificare: `F12 → Console`.
- **Resurse incarcate prin HTTP (nu HTTPS).** Mixed content; rezolvat automat pe GitHub Pages.

4.4 SEO

- **Document fara <title>.**
- **Lipsa <meta name="description">.**
- **Element <html> fara lang.**
- **Linkuri fara text descriptiv** (`<a href="X">click aici</a>` este penalizat).

5. Core Web Vitals

Cei trei indicatori folositi explicit de Google in algoritmul de ranking, incepand cu 2021:

- **LCP (Largest Contentful Paint)** – timpul pana la afisarea celui mai mare element vizibil. Prag: <2.5 secunde.
- **INP (Interaction to Next Paint)** – latentă între o acțiune a utilizatorului și răspunsul vizual. Prag: <200 milisecunde.
- **CLS (Cumulative Layout Shift)** – măsura instabilității vizuale în timpul încărcării. Prag: <0.1.

6. Optimizarea imaginilor

Imaginile reprezinta, in medie, 50% din dimensiunea unei pagini web. Optimizarea lor are impact direct asupra scorului Performance.

6.1 Format modern

WebP reduce dimensiunea cu 25–35% fata de JPEG, AVIF cu 50%. Conversia se face cu <https://squoosh.app> (interfata web, fara instalare).

6.2 Atribute necesare pe

✘ ASA NU

```

```

Probleme. Lipsește `alt` (a11y); lipsesc `width/height` (CLS); lipsește `loading` (download eager); o singură sursă pentru toate ecranele (banda risipită pe mobil).

✔ ASA DA – OPTIMIZAT COMPLET

```

```

Beneficii. Browserul alege rezoluția potrivită din `srcset`; imaginile sub fold sunt încărcate doar la nevoie; rezerva spațiu prin `width/height`; `alt` asigură accesibilitatea.

6.3 Lazy loading

Atributul `loading="lazy"` amână descărcarea imaginilor aflate sub *fold* (zona vizibilă inițial). Suportul browser este universal începând cu 2020.

Excepție: pentru imaginea *hero* (prima imagine din viewport), se folosește `loading="eager"` sau atributul lipsește; `loading="lazy"` aici întârzie LCP-ul.

7. Îmbunătățiri suplimentare pentru Performance

7.1 Preload pentru resurse critice

Resursele critice (font principal, imaginea hero) pot fi declarate cu `<link rel="preload">` pentru ca browserul să le descarce înainte de a parsa CSS-ul.

```
1 <link rel="preload" href="fonts/lato.woff2" as="font"
   type="font/woff2" crossorigin>
2 <link rel="preload" href="hero.webp" as="image">
```

7.2 CSS critic inline

Pentru paginile cu prima vizită prioritara (landing pages), CSS-ul critic (necesar pentru randarea zonei vizibile) poate fi inclus direct în `<head>` prin `<style>`, restul fiind încărcat asincron. Tehnica nu este necesară pentru pagini personale; merita atenție pentru site-uri comerciale.

Verificare cunoștințe

☰ PROVOCARE

1. Care sunt cele 4 categorii evaluate de Lighthouse?
2. Diferența între *lab data* și *field data*?
3. Care sunt cele 3 metrice Core Web Vitals și pragurile lor?
4. Spot the issue: `` pe o pagina de producție.
5. Când este corect să folosești `loading="lazy"` și când nu?

Recapitulare

✔ FAZA 6 – LIGHTHOUSE

- Patru categorii: Performance, Accessibility, Best Practices, SEO.
- Praguri: 90+ acceptabil, 95+ profesional, sub 50 necesită intervenție.
- Lab data (Lighthouse) vs field data (CrUX); PageSpeed Insights le combina.
- Executie: <https://pagespeed.web.dev> (web) sau Chrome DevTools → Lighthouse.
- Top fix-uri: `alt` pe imagini, `lang` pe `<html>`, contrast $\geq 4.5:1$, `width/height` pe `<img>`, `defer` pe `<script>`.
- Core Web Vitals: LCP $< 2.5s$, INP $< 200ms$, CLS < 0.1 .
- Imagini: WebP/AVIF, `srcset`, `loading="lazy"` (cu excepția imaginii hero), dimensiuni explicite.



🔧 APLICAȚIE PRACTICĂ

Platforma: PageSpeed Insights (<https://pagespeed.web.dev>).

Activitatea.

1. *Pasul 1 (3 minute).* Lipirea URL-ului paginii proprii deployate. Executarea analizei. Scorul tipic, după parcurgerea fazelor 1–5, este peste 90.
2. *Pasul 2 (5 minute).* Identificarea uneia sau a doua probleme cu impact ridicat. Aplicarea fix-ului în cod local. `git push`. Reexecutarea analizei după 60 de secunde.
3. *Pasul 3 (2 minute).* Aplicarea aceluiași audit pe varianta deployată a folder-ului `kit-starter/01-broken/`. Compararea scorurilor.

Continuare independentă: folder-ul `01-broken` conține 10 probleme documentate în fișierul README. Tinta este atingerea scorului 95+ pe toate cele patru categorii.

Durată estimată: 10 minute.

🎓 Resurse pentru aprofundare

- Google. *PageSpeed Insights*. <https://pagespeed.web.dev>
- Google. *Learn Performance*. <https://web.dev/learn/performance>

- Google. *Web Vitals*. <https://web.dev/articles/vitals>
- Google. *Lighthouse documentation*. <https://developer.chrome.com/docs/lighthouse/overview>
- WebAIM. *Contrast Checker*. <https://webaim.org/resources/contrastchecker/>
- Google. *Squoosh image optimizer*. <https://squoosh.app>
- Real Favicon Generator. <https://realfavicongenerator.net>

→ **Urmeaza:** Capitolul Resurse & urmatorii pași (cap. 7) propune un roadmap de 4 saptamani pentru consolidare și o lista curatata de tutoriale, comunitați și tool-uri pentru a continua singur.

Resurse & urmatorii pasi

Aplicatii interactive. Tutoriale. Referinte. Comunitati.

Acest capitol consolideaza resursele referentiate in capitolele anterioare si adauga sugestii pentru continuarea studiului dupa training. Materialele sunt grupate functional: aplicatii interactive (joculete), tutoriale structurate, referinte de cautare, comunitati, si tool-uri practice.

1. Roadmap pentru consolidare

📌 SAPTAMANA 1

Reconstruirea site-ului cu o tema personala (portofoliu, blog tematic, pagina pentru un eveniment). Folder-ul `kit-starter/02-reference/` serveste ca exemplu de implementare, nu ca sablon copiat direct.

📌 SAPTAMANA 2

Atingerea scorului 95+ pe toate cele patru categorii Lighthouse. Iteratia consta in audit – identificare a problemelor – remediere – reaudit.

📌 SAPTAMANA 3–4

Adaugarea unei functionalitati non-triviale: comutator pentru tema dark/light, formular cu trimitere efectiva (<https://formspree.io> ofera un endpoint gratuit), galerie de imagini cu lightbox, navigare cu mai multe pagini, animatii la scroll.

📌 LUNA 2 SI URMATOARELE

Inscrierea intr-un curriculum complet: *The Odin Project* sau *freeCodeCamp*. Ambele sunt gratuite si acopera traiectoria completa pana la dezvoltare full-stack, in 6–12 luni de studiu sustinut.

2. 🎮 Aplicatii interactive (joculete)

HTML / Accesibilitate

W3C Before / After Demo – comparatie intre o pagina inaccesibila si echivalentul sau accesibil.

CSS

Flexbox Froggy – <https://flexboxfroggy.com>

Grid Garden – <https://cssgridgarden.com>

CSS Diner – <https://flukeout.github.io> (selectori)

CSS Battle – <https://cssbattle.dev> (reproducerea unei imagini cu CSS minimal)

JavaScript

JS Robot – <https://lifesphere.eu/jsrobot>

Coding Fantasy – <https://codingfantasy.com> (lectiile JS sunt gratuite)

Codewars – <https://codewars.com> (probleme de tip kata)

Git / Deploy

Learn Git Branching – <https://learngitbranching.js.org>

Oh My Git! – <https://ohmygit.org> (joc desktop, gratuit, open source)

3. 📖 Tutoriale structurate

- **The Odin Project.** <https://www.theodinproject.com>
Curriculum complet, gratuit, parcurs estimat 6–12 luni. Acoperire: HTML, CSS, JavaScript, Git, Node.js, Ruby on Rails sau React.
- **freeCodeCamp.** <https://www.freecodecamp.org>
Certificari gratuite (Responsive Web Design, JavaScript Algorithms and Data Structures, Front End Development Libraries).
- **web.dev/learn.** <https://web.dev/learn>
Cursuri scurte produse de Google, actualizate frecvent: HTML, CSS, JavaScript, Performance, Forms, Accessibility, PWA.
- **javascript.info.** <https://javascript.info>
Manual extins pentru JavaScript, mai detaliat decat MDN, structurat didactic.
- **W3Schools.** <https://www.w3schools.com>
Referința sistematică cu „Try it Yourself” pe fiecare exemplu. Secțiuni separate: HTML, CSS, JS, TypeScript, React, SQL, Python.
- **Educative – Web Development: Unraveling HTML, CSS, JS.** <https://www.educative.io>
Curs comprehensiv (20h) cu pattern-ul „Hands-On / How it Works” – prezintă acțiunea, apoi explică.
- **Educative – HTML5 Canvas: From Noob to Ninja.** <https://www.educative.io>
Continuare pentru cei interesați de animații și grafică programatică (9h, peste 100 lecții).

4. 🏆 Pregătire pentru interviuri

- **GreatFrontend – 50 must-know HTML/CSS/JS interview questions.**
<https://www.greatfrontend.com/blog/50-must-know-html-css-and-javascript-interview-questions>
Lista celor mai frecvente întrebări tehnice la interviurile front-end, formulată de foști intervieviatori.
- **Frontend Interview Handbook.** <https://www.frontendinterviewhandbook.com>
Resursa free pentru pregătirea interviurilor front-end (sistem design, coding, comportamente).

- **LeetCode – JavaScript track.** <https://leetcode.com>
Probleme algoritmice si de structuri de date in JavaScript.

5. 🔍 Referinte de cautare

- **Mozilla Developer Network.** <https://developer.mozilla.org>
Sursa de referinta pentru standardele web. Cautarea se efectueaza cu pattern-ul „X mdn” pe orice motor de cautare.
- **Can I Use.** <https://caniuse.com>
Tabel detaliat pe browser si versiune pentru suportul fiecarui feature web.
- **CSS-Tricks.** <https://css-tricks.com>
Articole de detaliu pentru concepte CSS specifice; ghidul de flexbox si cel de grid sunt referinte standard.

6. 👥 Comunitati

- Subreddit-urile *r/webdev*, *r/learnprogramming*, *r/Frontend* – discutii zilnice si raspunsuri la intrebari concrete.
- **Stack Overflow.** <https://stackoverflow.com> – platforma de intrebari/raspunsuri tehnice. Primul rezultat pentru majoritatea erorilor cautate pe Google.
- **Frontend Mentor.** <https://www.frontendmentor.io> – provocari de tip design → implementare, cu peer review.
- Canalele Discord ale comunitatii BEST Iasi – pentru intrebari specifice contextului local.

7. 🛠 Tool-uri practice

- **Squoosh** – <https://squoosh.app> (optimizare imagini).
- **TinyPNG** – <https://tinypng.com> (compresie PNG / JPG).
- **Coolers** – <https://coolers.co> (generare palete de culori).
- **Google Fonts** – <https://fonts.google.com>.
- **Favicon Generator** – <https://favicon.io>.
- **Excalidraw** – <https://excalidraw.com> (diagrame, schite tehnice).

8. 💡 Idei de proiecte (pentru cand te plictisești)

Cele mai bune skill-uri se invata facand ceva ce te intereseaza pe tine. Cateva idei, de la 30 minute la cateva weekenduri:

Personale (pentru CV / share pe social)

- Portofoliu personal cu poza, sectiune „Despre”, proiecte, contact.
- Blog despre orice te intereseaza (gaming, anime, gatit, drumetii, fotografii).

- „Card de vizita” digital (CV web stylish, share-uibil prin URL).
- Pagina pentru un proiect BEST sau un eveniment universitar.

Distractive (1-2 ore fiecare)

- Tic-Tac-Toe sau Memory game in HTML/CSS/JS pur.
- Quiz interactiv cu intrebari preferate („Ce film e asta?”, „Ghicește tara dupa steag”).
- Pomodoro timer (25 min lucru / 5 min pauza, cu sunet).
- Calculator simplu, dar cu un design mișto.
- Generator de palete de culori la apasare de buton.
- Joc snake clasic, in canvas.

Utile (le folosești tu, prietenii tai)

- Habit tracker (verde la zilele cand ai facut treaba, roșu la cand n-ai).
- Lista de filme / serii / cărți de vazut, cu rating.
- Tracker de cheltuieli (carbohidrați, bani, ore studiate).
- Pagina pentru proiectul tau de școala (mai stylish decat un PDF).

Fan / pasiune

- Pagina dedicata serialului / anime-ului / formației tale preferate.
- Galerie cu pozele tale de la concerte / drumeții / vacanțe.
- Wiki personal pentru jocul tau favorit (Skyrim, Pokemon, etc.).

9. 🏆 Provocari pentru cand vrei sa te depășești

- „Site in 1 ora”. Cronometreaza-te. Pune un site decent online intr-o ora.
- „Lighthouse 100 / 100 / 100 / 100”. Niciun fix nu este de prisos.
- „30 zile de cod”. Un commit pe zi, orice marime, pe orice repo. Vezi cum se umple GitHub-ul de patrate verzi.
- „No CSS framework”. Refa unul dintre proiectele tale fara Bootstrap/Tailwind – doar CSS pur.
- „Cont GitHub cu 5 stele”. Construiește ceva de care alții vor sa puna stea. (Nu le cere stele – fa-le de la sine.)

• • •

📌 OBSERVATIE DE INCHEIERE

Slide-urile prezentarii si pagina personala construita in training partajeaza acelasi set de tehnologii (HTML, CSS, JavaScript). Functionalitatea *View Source* a oricarui browser ofera acces la codul sursa al oricarei pagini publice. Lectura acelui cod este, de acum inainte, accesibila.

Anexe

Glosar termeni. Checklist final. Erori comune si remediere.

Sectiunea finala consolideaza referintele rapide: definitiile termenilor folositi pe parcursul manualului, lista de verificari inainte de publicarea unui site, si problemele intalnite frecvent impreuna cu solutiile lor.

1. Glosar de termeni

Accesibilitate (a11y)

Practica de a crea continut utilizabil de catre persoane cu dizabilitati. Standardul de referinta este WCAG (Web Content Accessibility Guidelines).

API

Application Programming Interface. Set de functii prin care componentele software interactioneaza. In context web: `document.querySelector`, `fetch`, etc.

Arrow function

Sintaxa concisa pentru functii in JavaScript: `(a, b) => a + b`. Se comporta diferit fata de `function` privind `this`.

Cascade (CSS)

Algoritmul prin care browserul determina valoarea finala a unei proprietati cand mai multe reguli o vizeaza. Foloseste specificitate, ordine si importanta.

CDN

Content Delivery Network. Reteaua de servere distribuite geografic pentru servirea resurselor cu latenta minima.

CLS

Cumulative Layout Shift. Indicator Core Web Vitals pentru stabilitatea vizuala in timpul incarcarii.

Commit

Snapshot al starii fisierelor intr-un repository git, identificat printr-un hash unic.

CORS

Cross-Origin Resource Sharing. Mecanism de securitate care controleaza accesul la resurse din alte domenii.

CSS Grid

Model de layout bidimensional, complementar lui Flexbox; util pentru aranjamente in retea.

DOM

Document Object Model. Reprezentarea in memorie a documentului HTML, expusa programatic prin `document`.

Endpoint

URL specific care accepta cereri HTTP si returneaza un raspuns. Echivalent cu o „adresa” a unui serviciu.

Flexbox

Model de layout uni-dimensional in CSS pentru distribuirea spatiului pe o axa.

Hosting

Servirea fisierelor unui site catre vizitatori. GitHub Pages, Netlify si Vercel sunt servicii de hosting pentru site-uri statice.

HTTP

HyperText Transfer Protocol. Protocolul de baza al webului, prin care browserele cer si primesc resurse.

INP

Interaction to Next Paint. Indicator Core Web Vitals pentru receptivitate; latentă între interacțiune si feedback vizual.

JSON-LD

JSON for Linked Data. Format de date structurate folosit de motoarele de cautare pentru intelegerea continutului.

LCP

Largest Contentful Paint. Indicator Core Web Vitals; momentul afisarii celui mai mare element vizibil.

Lighthouse

Tool de audit automatizat dezvoltat de Google; evalueaza Performance, Accessibility, Best Practices, SEO.

NodeList

Colectie de noduri DOM returnata de `querySelectorAll`; iterabila prin `forEach`.

Open Graph (OGP)

Standard initiat de Facebook pentru previzualizarea paginilor pe retele sociale (`og:title`, `og:image`, etc.).

PAT

Personal Access Token. Token de autentificare pentru GitHub, folosit in locul parolei la operatiunile git.

Pseudo-clasa

Selector CSS care vizeaza o stare a unui element: `:hover`, `:focus`, `:checked`, `:first-child`.

Repository (repo)

Director versionat de git; contine codul, istoricul commit-urilor si configuratia.

Responsive design

Abordare prin care site-ul se adapteaza dimensiunii ecranului utilizatorului.

Schema.org

Vocabular standardizat pentru date structurate, suportat de motoarele de cautare majore.

Selector CSS

Pattern care identifica elementele HTML asupra carora se aplica o regula CSS.

Semantic HTML

Folosirea tag-urilor care descriu rolul continutului (`<header>`, `<nav>`) in locul `<div>` generic.

Specificitate

Mecanism prin care browserul rezolva conflictele intre reguli CSS care vizeaza acelasi element.

Static site

Site format din fisiere HTML/CSS/JS pre-generate, servite direct, fara procesare pe server.

Viewport

Zona vizibila a paginii in browser. Meta tag-ul `viewport` controleaza scalarea pe dispozitive mobile.

WCAG

Web Content Accessibility Guidelines. Standardul international pentru accesibilitatea continutului web.

2. Checklist de publicare

Lista de verificari aplicabila inainte de a considera un site finalizat. Se parcurge de sus in jos; orice item nebifat indica o lacuna remediabila.

HTML – structura

- ☐ Documentul incepe cu `<!DOCTYPE html>`.
- ☐ `<html>` are atributul `lang`.
- ☐ `<head>` include `<meta charset>`, `<meta viewport>`, `<title>` specific paginii.
- ☐ Structura folosesc tag-uri semantice (`<header>`, `<main>`, `<section>`, `<footer>`, `<nav>`).
- ☐ Un singur `<h1>` per pagina; ierarhia titlurilor nu sare niveluri.
- ☐ Toate imaginile au `alt`; cele decorative au `alt=""`.
- ☐ Toate input-urile au `<label for>` corespunzator.

CSS – aspect

- ☐ Reset minimal aplicat (`box-sizing: border-box, margin/padding default 0`).
- ☐ Variabilele CSS folosite pentru culori, spatiere, dimensiuni.
- ☐ Layout-ul rezista pe ecran 320px (test in DevTools – mobile preview).

- ☐ Media queries pentru ecrane $\geq 768\text{px}$.
- ☐ Contrast text/background $\geq 4.5:1$ pentru text normal.

JavaScript – comportament

- ☐ `<script>` are atributul `defer`.
- ☐ Niciun `var`; doar `const` și `let`.
- ☐ Comparatii doar cu `===`.
- ☐ Console-ul (F12) nu afiseaza erori la incarcarea paginii.
- ☐ Form-urile interceptate cu `e.preventDefault()` (sau au `action valid`).

SEO + Open Graph

- ☐ `<title>` specific, 50–60 caractere.
- ☐ `<meta name="description">` 150–160 caractere.
- ☐ Cinci tag-uri `og::` type, title, description, image, url.
- ☐ `og:image` este URL absolut, dimensiune $\geq 1200 \times 630$.
- ☐ `<meta name="twitter:card" content="summary_large_image">` prezent.
- ☐ Favicon (`<link rel="icon">`) configurat.

Deploy + Lighthouse

- ☐ Site accesibil pe URL public (`username.github.io/repo`).
- ☐ `.gitignore` prezent in repository.
- ☐ Lighthouse Performance ≥ 90 .
- ☐ Lighthouse Accessibility ≥ 95 .
- ☐ Lighthouse Best Practices ≥ 95 .
- ☐ Lighthouse SEO ≥ 95 .
- ☐ Previzualizare validata pe `opengraph.xyz`.

3. Erori comune si remediere

Lista neexhaustiva de probleme intalnite frecvent de incepatori, cu cauza si solutia.

HTML / CSS

- **Diacriticele apar ca moză.** Cauza: lipsa `<meta charset="UTF-8">` sau fisier salvat in alta codificare. Solutie: adaugarea meta tag-ului si salvarea fisierului in UTF-8 din editor.
- **Stilurile CSS nu se aplica.** Cauze: (1) calea `href` incorecta in `<link>`; (2) numele clasei diferit fata de selector; (3) regula are specificitate mai mica decat alta. Verificare: DevTools → Inspect → tab *Styles*.

- **Pe mobil pagina apare mica/dezordonat.** Cauza: lipsa `<meta name="viewport">`.
- **Flexbox-ul nu functioneaza.** Cauza: `display: flex` aplicat pe element gresit; trebuie pe *containerul* parinte, nu pe copii.

JavaScript

- `querySelector` returns `null`. Cauze: (1) selectorul nu corespunde niciunui element; (2) script-ul ruleaza inainte de parsarea HTML-ului (lipseste `defer`); (3) typo in numele clasei/id-ului.
- `addEventListener` is not a function. Cauza: rezultatul `querySelector` este `null`. Verificare: `console.log(el)` inainte de `addEventListener`.
- **Form-ul reincarca pagina la submit.** Cauza: lipsa `e.preventDefault()` in handler.
- `console.log` not defined. Foarte rar; de obicei o tipografica (`Console.log` cu C mare).

Git / Deploy

- `git push` cere parola si o respinge. Cauza: GitHub a inlocuit autentificarea prin parola cu PAT. Solutie: generare PAT la <https://github.com/settings/tokens> cu scope `repo`; folosirea tokenului in locul parolei.
- **Pages returneaza 404 dupa push.** Cauze: (1) intervalul de propagare (~60s); (2) fisierul de pornire nu se numeste `index.html` (case-sensitive); (3) cache-ul browserului – `Ctrl+F5` pentru hard refresh.
- `fatal: not a git repository`. Cauza: comanda rulata in afara unui director versionat. Solutie: `cd` in directorul corect sau `git init -b main`.
- **Modificarile nu apar pe Pages.** Cauza: nu s-a executat `git push` dupa `commit`; sau cache-ul Pages – asteptare ~60s.

SEO + Lighthouse

- **Card-ul WhatsApp nu afiseaza imaginea.** Cauza principala: `og:image` are URL relativ in loc de absolut. Verificare: opengraph.xyz raporteaza statusul.
- **Lighthouse Performance scor mic.** Cauze frecvente: (1) imagini nedimensionate, supradimensionate; (2) script-uri sincrone in `<head>`; (3) format imagini neoptimizat (PNG/JPEG cu fisiere mari).
- **Lighthouse Accessibility 50–80.** Cauze frecvente: (1) `alt` lipsa; (2) contrast scazut; (3) lipsa `lang` pe `<html>`; (4) butoane fara nume accesibil.